

Co-Evolving LLM Decision and Skill Bank Agents for Long-Horizon Tasks

Anonymous ACL submission

Abstract

Long-horizon interactive environments are a testbed for evaluating agents’ skill usage abilities. These environments demand multi-step reasoning, the chaining of multiple skills over many timesteps, and robust decision-making under delayed rewards and partial observability. Games are a good testbed for evaluating agent skill usage in environments. Large Language Models (LLMs) offer a promising alternative as game-playing agents, but they often struggle with consistent long-horizon decision-making because they lack a mechanism to discover, retain, and reuse structured skills across episodes. We present **COS-PLAY**, a co-evolution framework in which an LLM decision agent retrieves skills from a learnable skill bank to guide action taking, while an agent-managed skill pipeline discovers reusable skills from the agent’s unlabeled rollouts to form a skill bank. Our framework improves both the decision agent to learn better skill retrieval and action generation, while the skill-bank agent continually extracts, refines, and updates skills together with their contracts. Experiments across six game environments show that **COS-PLAY** with an 8B base model achieves over 25.1% average reward improvement against four frontier LLM baselines on single-player game benchmarks while remaining competitive on multi-player social reasoning games.

1 Introduction

A central goal of artificial general intelligence is to build autonomous agents that improve through interaction by discovering reusable *skills*, *i.e.*, reusable, temporally extended behavior protocols extracted from trajectories, and invoking them to solve increasingly complex tasks. This paradigm has a long history, spanning self-improving game agents in Atari and AlphaZero (Mnih et al., 2013; Silver et al., 2017) to modern LLM agents for coding, web interaction, and embodied exploration (Wang et al., 2025b; He et al., 2025; Xiang

et al., 2025; Huang et al., 2025a). Across these settings, self-learning has expanded from structured games to richer interactive environments, while the core explore–learn–reuse loop remains fundamental to autonomous improvement. As agents operate in increasingly complex domains, the challenge shifts from merely acquiring useful skills to organizing, refining, and retrieving them so they can be reused reliably across future tasks.

A natural way to address this challenge is to maintain an external *skill bank*: a structured library of reusable behavior protocols that the agent can acquire, refine, and compose over time (Li et al., 2026a). Recent work increasingly treats *skills* as a distinct abstraction layer for agentic LLMs beyond one-off tool calls or prompts, since skills package reusable procedural knowledge together with execution guidance and applicability constraints (Xu and Yan, 2026; Jiang et al., 2026b). Rather than repeatedly retraining the backbone LLM, the system improves by updating this skill bank and learning to invoke the right skills at the right time. Yet these two components are tightly coupled: a skill bank is only useful if the decision agent can select and execute its skills effectively, and the decision agent is only as capable as the skills available to it. This interdependence motivates two questions: **(Q1)** Should the skill bank and decision agent be learned *jointly*, rather than separately? **(Q2)** What properties make a skill bank useful for improving decision-making?

To study these questions, we need an environment that reveals co-evolution dynamics: episodes must be long enough to require *multiple* skills in sequence, subgoals must *recur* across episodes so that reuse is meaningful, and rollouts must be cheap enough to support iterative skill-bank refinement. Games offer such a setting in a safe and reproducible sandbox (Wang et al., 2023; Raad et al., 2024; Berner et al., 2019; Xu et al., 2025b), while still demanding long-context understanding,

memory, and adaptive decision-making (Hu et al., 2024). At the same time, they pose challenges that make naive approaches insufficient: delayed rewards (Liu et al., 2025), compositional strategies, and limited high-quality demonstrations or skill annotations (Liao et al., 2025; Lu et al., 2025). Consequently, many existing game-playing agents still rely on curated human data (Hu and Sadigh, 2023) or prolonged training with external feedback, whereas we instead aim to learn a co-evolving skill bank directly from interaction.

In this paper, we propose a **multi-agent co-evolution** framework for long-horizon game playing. The framework comprises: (i) an LLM-based **decision agent** that maintains an intention and active skill, retrieves or switches skills as needed, and executes primitive actions conditioned on both; and (ii) a **skill bank agent** that performs unsupervised skill discovery and maintenance by segmenting trajectories, learning compact skill contracts, and refining the skill bank over time. The two components improve each other in a closed loop: the decision agent uses the current skill bank for multi-step, skill-guided decision-making, while the skill bank agent converts newly collected rollouts into better reusable skills. We optimize both agents with Group Relative Policy Optimization (GRPO) (Feng et al., 2025): the decision agent is trained to improve skill retrieval and action execution, while the skill bank agent is trained for skill segmentation, contract learning, and bank curation. To our knowledge, this is one of the first frameworks to couple LLM-based game decision-making with an agentic skill-bank pipeline for unsupervised skill discovery and continual refinement in a unified co-evolution loop. Our main contributions are:

- We propose a co-evolution framework for long-horizon gameplay that closes the loop between decision making and skill learning: an LLM-based decision agent interacts with the environment to collect trajectories, and a skill bank agent converts these rollouts into reusable skills that improve future decision making;
- We provide a comprehensive evaluation of COS-PLAY across six game environments requiring multi-hop skill usage, covering both task performance and skill reusability. Built on an 8B base model, COS-PLAY achieves over **25.1%** average gain against four frontier LLM baselines on single-player games while

remaining competitive with SOTA LLMs on multi-player social reasoning tasks.

2 Related Work

Agents for Game Playing and Benchmarks.

Game agents and benchmarks are useful for studying long-horizon decision-making because they stress memory, planning, and real-time action under delayed rewards and complex dynamics. However, despite rapid progress, relatively few works study game-playing agents at scale due to the difficulty of interactive environments and evaluation. BALROG (Paglieri et al., 2024), VS-Bench (Xu et al., 2025a), VideoGameBench (Zhang et al., 2025), and VisGym (Wang et al., 2026) introduce increasingly challenging benchmarks for long-horizon reasoning, strategic interaction, and multi-step visual decision-making, revealing large gaps between frontier models and human performance. On the method side, Optimus-1 (Li et al., 2024), VARP (Chen et al., 2024), and AVA (Ma et al., 2025) develop agents for Minecraft, ARPGs, and StarCraft II, highlighting the importance of skill exploration, memory, and planning in complex gameplay (Zhai et al., 2025). These advances motivate our framework, which uses games as a controlled setting for co-evolving decision policies with a refineable skill bank. Rather than focusing only on benchmarks or game-specific agents, we study how reusable skills can be extracted from trajectories, refined over time, and fed back to improve long-horizon decision-making across diverse games.

Memory and Skill-Augmented Self-Improving Agents.

Memory- and skill-augmented agents have gained attention because long-horizon decision-making benefits from reusing past experience and executable skills instead of solving each step from scratch, especially in complex multi-modal tasks. Recent work improves self-evolving agents through memory and skill reuse (Li et al., 2026a,b; Huang et al., 2025a; Li et al., 2025). PolySkill (Yu et al., 2025b) improves skill transfer by decoupling abstract goals from concrete implementations. SAGE (Wang et al., 2025a), SkillRL (Xia et al., 2026), SCALAR (Zabounidis et al., 2026), and XSkill (Jiang et al., 2026a) use skill-augmented learning to improve skill generation, grounding, composition, retrieval, and policy performance. Memweaver (Yu et al., 2025a) introduces a hierarchical dual-memory framework combining graph-based retrieval with LLM-

summarized cognitive memory. ProcMEM (Mi et al., 2026) and CASCADE (Huang et al., 2025b) emphasize procedural memory and cumulative skill creation. UI-Mem (Xiao et al., 2026) studies hierarchical experience memory for long-horizon online RL, while MemRL (Zhang et al., 2026b) and MemSkill (Zhang et al., 2026a) treat memory retrieval and operations as runtime mechanisms for improving frozen agents. These works motivate our framework mostly focus on memory augmentation, skill transfer, or runtime retrieval within largely fixed pipelines. In contrast, our method studies co-evolution, where a decision agent and skill bank improve each other over time by automatically extracting, refining, and reusing skills from unlabeled trajectories to improve downstream decision-making.

3 Problem Formulation

Preliminary. We consider an interactive game environment $\mathcal{E} = (\mathcal{O}, \mathcal{A}, P, R, \gamma, T)$ with horizon T , where $\mathcal{O}$ is the observation space, $\mathcal{A}$ is the action space, $P(o_{t+1} \mid o_t, a_t)$ denotes the environment transition dynamics, $R(o_t, a_t, o_{t+1}) \in \mathbb{R}$ is the reward function, and $\gamma \in (0, 1]$ is the discount factor. At each timestep t , an agent observes $o_t \in \mathcal{O}$, takes an action $a_t \in \mathcal{A}$, receives reward $r_t \in \mathbb{R}$, and transitions to the next observation o_{t+1} . A trajectory (episode) is denoted by $\tau = (e_1, e_2, \dots, e_T)$, where each step is represented as an experience

$$e_t = (o_t, a_t, r_t, o_{t+1}, d_t, z_t)$$

Here $d_t \in \{0, 1\}$ is the termination flag, and z_t is the agent’s latent intention state at timestep t , representing its internal strategic interpretation of the environment (Qi and Zhu, 2018; Wu et al., 2023). This latent intention may correspond to either a short-horizon action intention for the next step or a higher-level strategic skill that guides behavior over the next several steps. In our framework, z_t is generated from the agent’s understanding of the current environment state and is iteratively updated throughout the trajectory as the agent interacts with the environment and refines its strategic objective.

Skills. A skill s_k is a reusable, temporally extended behavior abstraction extracted from trajectory segments. Each skill is stored in the skill bank as a structured *skill protocol* with the following components: *Summary*, which describes the skill’s purpose; *Pre-condition*, which specifies when the skill is applicable; *Plan*, which outlines how the

skill should be carried out; *Success/Abort Criteria*, which indicate when execution should terminate successfully or be abandoned; and *Contract*, which summarizes the state changes the skill is expected to produce. These components form the skill’s externally queryable interface for retrieval and execution, while the skill itself remains grounded in the set of supporting trajectory segments from which the protocol and contract are learned and refined over time.

Refineable Skill Bank. A key property of our framework is that skills are not fixed after initialization. Instead, as interactive learning between the agents and the environment continues, the skill bank is continuously updated and refined with new trajectory evidence. As new rollouts are collected, the skill bank agent updates the bank by segmenting trajectories, refining existing skills, and storing emerging skills.

Skill-augmented Decision-making. Given the current skill bank $\mathcal{B}$, the decision agent maintains an intention state z_t that captures its current strategic focus and skill-level subgoal. At each timestep, it first retrieves candidate skills from the bank, selects one based on the current observation and intention, updates its intention conditioned on the selected skill, and finally executes an action. Formally,

$$\tilde{s}_t = \pi_{\theta}^{\text{skill}}(o_t, \mathcal{B}), \quad (1)$$

$$z_t = \pi_{\theta}^{\text{int}}(o_t, \tilde{s}_t), \quad (2)$$

$$a_t \sim \pi_{\theta}^{\text{act}}(\cdot \mid o_t, z_t, \tilde{s}_t). \quad (3)$$

We model the decision agent with three components serving distinct roles: $\pi_{\theta}^{\text{skill}}$ for skill retrieval-and-selection, $\pi_{\theta}^{\text{int}}$ for intention updating, and $\pi_{\theta}^{\text{act}}$ for action execution. Here, $\tilde{s}_t$ denotes the retrieved skill or skill set at timestep t , and z_t denotes the updated intention state conditioned on the current observation and retrieved skill. The agent is trained to maximize the expected cumulative reward:

$$\max_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T r_t \right]. \quad (4)$$

Skill Bank Update. We formulate skill-bank refinement iteratively because the skill bank and decision agent co-evolve over training: each round of newly collected trajectories reflects the current policy, while the updated bank shapes subsequent skill retrieval and action execution. The trajectory set $\mathcal{D}^{(u+1)}$ collected up to co-evolution iteration

$u + 1$ is passed to the skill bank agent to refine the current skill bank $\mathcal{B}^{(u)}$:

$$\mathcal{B}^{(u+1)} = \Phi_S(\mathcal{B}^{(u)}, \mathcal{D}^{(u+1)}), \quad (5)$$

where $\mathcal{B}^{(u)}$ and $\mathcal{B}^{(u+1)}$ denote the skill bank before and after the update, $\mathcal{D}^{(u+1)}$ denotes the accumulated rollout set, and Φ_S denotes the skill-bank update pipeline. This pipeline continually updates the skill bank so that retrieved skills remain reusable and aligned with the evolving decision policy.

4 Methodology

We propose a co-evolving multi-agent framework for long-horizon video-game decision-making via unsupervised trajectory decomposition and skill-bank refinement (Figure 1). The framework has two components: **(a) Decision Agent A_D** : an LLM-based agent that interacts with the game through primitive actions and skill retrieval. At each step, it summarizes the current state, retrieves relevant skill candidates from the skill bank, updates its intention, selects or switches skills when needed, and executes an action. **(b) Skill Bank Agent A_S** : an LLM-based pipeline that converts unlabeled trajectories into reusable protocol-based skills and learns compact effect contracts for them. It updates the skill bank by proposing new skill candidates, refining low-quality skills, and revising skill protocols over time. Together, the decision agent generates trajectories, and the skill bank agent transforms them into structured skills that support future decisions through skill retrieval and selection.

4.1 Skill-Augmented Decision Agent

We design a skill-augmented decision agent that unifies skill retrieval, intention updating, action execution, and reward-based learning for long-horizon gameplay. At each step, it maintains a state summary, intention, and active skill; either continues the current skill or retrieves a new one; and executes a primitive action conditioned on the observation, intention, and skill plan. The resulting observation and reward are used to update the agent state, while training is driven by a composite reward combining environment feedback, skill-following shaping, and switching cost.

Skill Retrieval and Intention Update. The decision agent maintains an active skill and a current intention. Given the current state summary, it either continues the active skill or retrieves a new

one from the skill bank when the current skill is unavailable, exhausted, or ineffective. Each skill is a structured context prompt containing protocol steps, preconditions, and termination cues. After each primitive action, the agent updates a short natural-language intention tag summarizing its immediate subgoal. When this intention shifts sharply, it signals a subtask transition and triggers skill switching (Konidaris et al., 2010).

Action Execution. At each timestep, the agent executes a primitive environment action conditioned on the current state summary, task context, recent interaction history, the current intention, and, when available, the active skill plan. The active skill does not replace low-level control, but instead guides action generation toward coherent multi-step behavior. The executed action produces the next observation and reward, which are then used to update the agent state and intention.

4.2 Skill Bank Agent for Skill Discovery and Maintenance

With decision agent rollouts, a skill bank agent analyzes the trajectory to extract reusable skills and maintain the skill bank over time. This agent has a four-stage pipeline: proposing candidate skill boundaries, inferring a segmentation with skill labels, learning and verifying effect contracts, and finally updating the bank through refinement, materialization, merging, splitting, and retirement. In this way, the bank evolves from repeated trajectory evidence while remaining compact, stable, and useful for downstream decision making. Figure 2 walks through the full pipeline on a single Diplomacy episode.

Boundary Proposal. Boundary proposal is a heuristic candidate-generation step that identifies plausible skill-transition points in a trajectory. For each timestep, we compute a boundary score from lightweight local signals that often indicate a change of skill, including predicate flips between adjacent states, intention-tag changes, reward or event spikes, optional surprisal peaks, and transitions between primitive-action execution and skill-selection steps. Timesteps with high scores are retained as candidate cut points, after which nearby candidates are merged to remove redundancy. This produces a compact, high-recall boundary set that is later disambiguated by the segmentation module.

Infer Segmentation. Starting from the candidate boundaries, we choose the subset that best explains the trajectory as a sequence of skill segments. Each

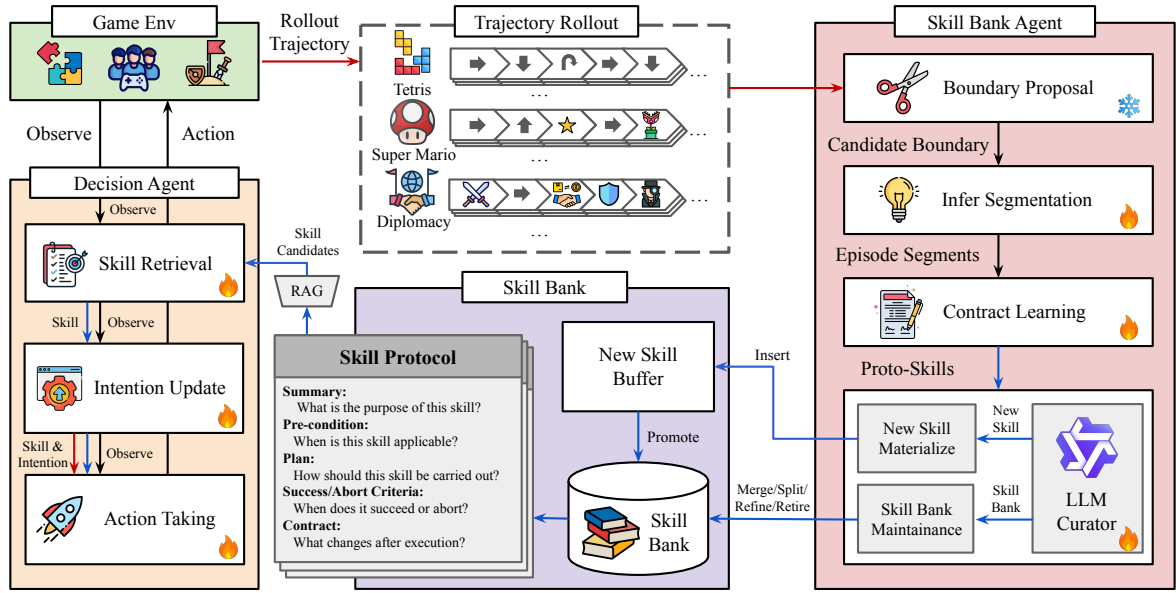


Figure 1: **Overview of COS-PLAY.** COS-PLAY is a multi-agent co-evolution framework that couples gameplay with skill learning. It consists of a decision agent (Orange Box), a skill bank agent (Red Box), and a skill bank (Purple Box). The decision agent interacts with the game by retrieving skills, updating intentions, and selecting actions. After each episode, the skill bank agent segments trajectories, learns skill contracts, adds new skills, and updates existing ones through refinement, merging, splitting, or retirement. This loop enables the agent to acquire reusable skills from prior experience and continually improve its policy.

segment is summarized by its observed effects and compared against skills in the current bank. We first score candidate skills by overlap between the segment’s added and deleted predicates and each skill’s learned *effect contract*, *i.e.*, a compact specification of the state changes that the skill reliably produces when executed successfully, with a small bonus for stronger prior support, and then re-rank the top candidates using the skill bank agent. If a segment matches an existing skill with high confidence, we assign that label. Otherwise, we mark it as *new skill*, indicating behavior not yet represented in the bank. Its observed effects are kept as provisional evidence, and a reusable effect contract is learned later by aggregating and verifying multiple such segments in the contract learning stage.

Contract Learning. For each skill, we aggregate the added and deleted predicates across its decoded segments to learn an *effect contract* that captures its reliable state changes. We keep only consensus effects that appear consistently across instances, treating infrequent ones as noise. When enough evidence is available, the contract can be enriched by an LLM-based summarization module, whose suggestions are added to the statistical consensus. We then verify whether the contract effects are supported by observed state changes, and only contracts with sufficiently high pass rates

are written back into the skill bank. These verified contracts also provide a compatibility prior for segment decoding, giving higher confidence to segments whose observed effects better match a skill’s contract.

Skill Bank Maintenance. The final stage mutates the skill bank through five operations: *refine*, *materialize*, *merge*, *split*, and *retire*. Segments labeled as *new skill* are first evaluated for consistency and reuse potential rather than added immediately; promising ones enter a new-skill buffer, where similar candidates are grouped until sufficient evidence supports materializing a new bank entry. For existing skills, *refine* updates contracts when new verified evidence meaningfully changes their reliable effects, *merge* removes redundant near-duplicate skills with highly overlapping contracts, and *split* marks overly broad or inconsistent skills for later re-segmentation. We also *retire* skills that are no longer used or supported by recent trajectories, keeping the bank compact and relevant. The skill bank agent may approve or reject proposed mutations before they are committed to the bank.

4.3 Co-Evolution Framework

We adopt a *co-evolution* framework in COS-PLAY where the decision agent and skill bank agent are mutually dependent. The decision agent uses the

current skill bank to interact with the environment and collect trajectories; the skill bank agent then segments these into reusable skills, infers contracts, and refines the bank. This closed loop is self-reinforcing: better skills improve decision making, and better rollouts improve skill learning. Both agents are updated via GRPO (Shao et al., 2024) using separate LoRA adapters (Hu et al., 2022). The decision agent has two adapters: *action-taking*, which rewards progress on the active skill while discouraging unnecessary retrieval, and *skill-retrieval*, which is evaluated at episode end and rewards skills that led to useful, contract-satisfying outcomes. This decomposition lets the agent jointly improve *what* skill to retrieve and choose and *how* to execute it. The skill bank agent uses three adapters. The *segmentation* adapter maps high-value trajectory segments to existing skills while allowing new skills to appear. The *contract* adapter learns compact state-transition summaries that generalize across episodes. The *curator* adapter updates the bank through retention, exploration, and evidence-based merging, splitting, or rejection. Full reward definitions are given in Appendix G.

5 Experiments

Evaluations. We evaluate on six game environments spanning single-player puzzle and interactive control tasks, including 2048, Candy Crush, Tetris, and Super Mario Bros. (Hu et al., 2025; Park et al., 2025), as well as multi-player social reasoning games, Avalon and Diplomacy (Zhai et al., 2025). Avalon is a team-based hidden-role game in which only one side can win. It is especially challenging for the Good side, which must infer hidden roles from sparse signals such as proposals, votes, and quest outcomes, while Evil players begin fully coordinated and can strategically conceal or sabotage (Light et al., 2023). Diplomacy further increases the difficulty by requiring long-context reasoning, negotiation, alliance tracking, and multi-phase planning. Smaller LLMs have recently been shown to struggle with both Avalon and Diplomacy due to their highly social interactive nature, making them an excellent testbed for COS-PLAY (Rahimi-rad et al., 2025; Duffly et al., 2026).

Across all environments, observations are converted into structured textual states, and the agent interacts through discrete text actions under a unified Gym-style API (Brockman et al., 2016). Full environment details, including observations, ac-

tions, horizons, and rewards, are provided in Appendix B.

Cold Start Initialization. We use GPT-5.4 as a teacher model to generate 60 seed trajectories per game. We then apply supervised fine-tuning (SFT) on these trajectories to train a Qwen3-8B (Yang et al., 2025) model, which serves as the shared initialization for both the decision agent and the skill bank agent.

Co-Evolving Training Setting. The co-evolving training proceeds in three stages: (1) the decision agent interacts with the environment to collect rollouts; (2) the skill-bank agent segments skill candidates from these rollouts and updates the skill bank; and (3) GRPO updates both agents. Starting from the cold-start model, we attach a separate LoRA adapters (Hu et al., 2022) for each trainable function: two for the decision agent (skill retrieval and action taking) and three for the skill bank agent (infer segmentation, contract learning, and skill bank maintenance). We discuss this multi-adapter design further in Appendix F. For social reasoning games (Avalon and Diplomacy), we use GPT-5-mini (OpenAI, 2026a) as opponents to provide strong supervision signals during evolution. Full training details and training curves are in Appendix C and I.

Reward Design and Metrics. For single-player games, we report the native game rewards provided by the benchmark environments (Hu et al., 2025; Park et al., 2025). For social games, we use team win rate for Avalon and the number of occupied supply centers for Diplomacy as the main evaluation metrics, following (Zhai et al., 2025), to provide a consistent basis for comparison across baselines. More details about reward design are included in Appendix G.

5.1 Main Results

We compare COS-PLAY with strong frontier LLMs, including GPT-5.4 (OpenAI, 2026b), GEMINI-3.1-PRO (Google DeepMind, 2026), CLAUDE-4.6-SONNET (Anthropic, 2026), and GPT-OSS-120B (OpenAI, 2025). Results are shown in Table 1. General reasoning and qualitative results appear in Appendix H and K.

COS-PLAY matches or exceeds frontier LLMs with efficient, few-shot adaptation. Overall, COS-PLAY achieves a substantial average improvement of 25.1% over GPT-5.4 on single-player games over 16 runs. More importantly, COS-

PLAY requires only few-shot adaptation to transfer to new benchmarks: each game needs at most 25 iterations to reach strong performance, compared with the hundreds of training steps often required by prior RL-based game agents (Shu et al., 2021), suggesting the strong data efficiency, learnability, and cross-domain adaptability of our framework.

A small model with structured state tracking and reusable skills can approach frontier LLMs on social reasoning tasks that require long-context memory and multi-turn consistency. For multi-player social games, we use GPT-5.4 as the opponent model, against which most non-GPT baselines underperform (Table 1). We provide role-wise and power-wise breakdowns in Table 5 and Table 6 in Appendix D. In Avalon, COS-PLAY is comparable to GEMINI-3.1-PRO and GPT-OSS-120B, trailing by only 1% in win rate, while in Diplomacy it outperforms GEMINI-3.1-PRO by 8.8%. These results suggest that structured state tracking and reusable skills provide an effective inductive bias for long-horizon social reasoning, even with a much smaller backbone model.

5.2 Ablation Study

We compare our full pipeline against several variants in Table 1: the base QWEN3-8B (Yang et al., 2025); the SFT decision agent without a skill bank (SFT w/o SKILL); the SFT model with the first skill bank from the initial co-evolution iteration (SFT + 1ST SKILL– No Co-Evolution); the SFT model with the final skill bank (SFT + FINAL SKILL– No Co-Evolution); the GRPO decision agent without a skill bank (GRPO w/o SKILL); and the GRPO decision agent with the first skill bank (GRPO + 1ST SKILL). The results show that no single component is sufficient. While some variants improve specific games, their gains are inconsistent across domains. SFT w/o SKILL improves action formatting but lacks reusable long-horizon structure, whereas GRPO w/o SKILL improves behavior yet remains unstable under sparse rewards. Variants with mismatched skill banks perform worse, as the policy and skill bank are optimized for different state distributions, making retrieved skills less aligned and often harmful. These findings suggest that the main advantage comes not from skills or RL alone, but from co-evolution, which keeps the skill bank aligned with the policy’s changing behavior and task distribution.

5.3 Skill Reusability Analysis

Apart from decision performance, we also evaluate the *reusability* of the learned skills. Unlike prior approaches that manually specify skills or distill them from strong LLMs with human supervision, our skill-bank agent automatically extracts skills from unlabeled rollout trajectories. A useful skill should both improve downstream rewards and remain reusable across episodes, since repeated reuse suggests stable preconditions, effects, and execution protocols rather than memorization of isolated trajectories. As summarized in Table 2 and illustrated in Figure 3, our skill bank learns compact and adaptable skill abstractions. In the Diplomacy case study, the bank grows in strategic diversity over training while remaining compact through periodic merge-and-split curation. The x-axis in Figure 3 (left) corresponds to co-evolution iterations, where each point reflects the skill bank after one round of rollout collection and bank update.

5.4 Discussion

Few-Shot Adaptation with Minimal Loss in General Reasoning. Table 1 shows that COS-PLAY substantially improves game-playing performance with only a few adaptation iterations while largely preserving the base model’s general reasoning ability. On benchmarks such as Math-500 and MMLU-Pro (Appendix H), performance drops remain small, suggesting that gains in game environments do not come at the cost of severe reasoning degradation, even though the adapted 8B model reaches strong performance within 25 co-evolution steps.

Why Reusable Skills Matter in Long-Horizon Tasks. Table 2 suggests that COS-PLAY learns reusable skill abstractions rather than memorizing isolated trajectories. This is especially important in long-horizon and socially strategic settings, where coherent multi-step behavior is required. By retrieving only a small set of relevant skills, COS-PLAY provides compact guidance that improves action selection with low overhead, while continual refinement supports stable co-evolution and efficient adaptation.

6 Conclusion

We present COS-PLAY, a co-evolution framework in which an LLM decision agent retrieves skills from a skill bank to guide action taking, while an agent-managed skill pipeline discovers reusable

Model	Single-Player					Multi-Player	
	2048 Reward $\uparrow$	Tetris Reward $\uparrow$	CandyCrush Reward $\uparrow$	Super Mario Bros Reward $\uparrow$	Avg. Reward $\uparrow$	Avalon Win Rate $\uparrow$	Diplomacy Mean SC $\uparrow$
GPT-5.4	1126.6 $\pm$ 150.2	458.2 $\pm$ 203.5	532.6 $\pm$ 24.8	752.0 $\pm$ 35.7	717.4	41.2 $\pm$ 10.6	4.70 $\pm$ 0.35
GEMINI-3.1-PRO	813.3 $\pm$ 143.6	372.7 $\pm$ 157.7	334.3 $\pm$ 59.4	436.8 $\pm$ 86.1	489.3	42.0 $\pm$ 13.2	2.72 $\pm$ 0.26
CLAUDE-4.6-SONNET	945.0 $\pm$ 134.5	444.2 $\pm$ 182.6	328.6 $\pm$ 23.8	399.5 $\pm$ 53.4	529.3	40.0 $\pm$ 13.1	3.16 $\pm$ 0.19
GPT-OSS-120B	1029.5 $\pm$ 122.0	358.1 $\pm$ 139.7	334.4 $\pm$ 40.5	968.5 $\pm$ 175.0	672.6	40.0 $\pm$ 13.1	2.46 $\pm$ 0.25
QWEN3-8B	131.0 $\pm$ 102.6	32.0 $\pm$ 8.5	519.9 $\pm$ 37.8	835.5 $\pm$ 161.6	379.6	30.0 $\pm$ 9.9	2.64 $\pm$ 0.18
SFT w/o SKILL	516.7 $\pm$ 172.3	28.2 $\pm$ 9.7	356.1 $\pm$ 30.1	736.8 $\pm$ 130.4	409.5	28.7 $\pm$ 9.7	2.75 $\pm$ 0.13
SFT + 1ST SKILL	385.5 $\pm$ 239.7	35.8 $\pm$ 7.0	569.6 $\pm$ 29.5	871.9 $\pm$ 126.2	465.7	21.2 $\pm$ 8.9	2.89 $\pm$ 0.20
SFT + FINAL SKILL	64.8 $\pm$ 46.0	24.4 $\pm$ 7.0	554.4 $\pm$ 24.3	794.4 $\pm$ 112.9	359.5	25.0 $\pm$ 9.3	2.65 $\pm$ 0.25
GRPO w/o SKILL	510.0 $\pm$ 249.5	96.7 $\pm$ 30.3	163.3 $\pm$ 71.4	669.4 $\pm$ 130.1	359.9	36.2 $\pm$ 10.3	2.76 $\pm$ 0.19
GRPO + 1ST SKILL	152.0 $\pm$ 107.9	93.7 $\pm$ 37.7	353.8 $\pm$ 20.2	621.2 $\pm$ 130.2	305.2	31.2 $\pm$ 10.0	2.56 $\pm$ 0.22
GRPO NO-BANK	854.2 $\pm$ 134.2	283.9 $\pm$ 98.2	578.0 $\pm$ 13.6	753.4 $\pm$ 137.5	–	20.0 $\pm$ 10.9	2.61 $\pm$ 0.19
GRPO FIXED-BANK	898.5 $\pm$ 30.3	238.1 $\pm$ 85.7	505.7 $\pm$ 27.4	770.5 $\pm$ 87.0	–	22.0 $\pm$ 11.2	2.54 $\pm$ 0.19
GPT-5.4 + FINAL SKILL	1176.2 $\pm$ 168.2	419.4 $\pm$ 150.8	581.2 $\pm$ 33.2	933.7 $\pm$ 133.5	–	41.2 $\pm$ 10.8	4.77 $\pm$ 0.28
MEMSKILL (ZHANG ET AL., 2026A)	823.1 $\pm$ 376.5	247.6 $\pm$ 81.5	498.0 $\pm$ 50.5	307.5 $\pm$ 33.2	–	26.0 $\pm$ 11.8	2.35 $\pm$ 0.31
MEMRL (ZHANG ET AL., 2026B)	778.1 $\pm$ 344.1	10.9 $\pm$ 0.8	100.7 $\pm$ 24.4	975.4 $\pm$ 93.0	466.3	36.0 $\pm$ 12.9	2.76 $\pm$ 0.17
SKILLRL (XIA ET AL., 2026)	963.6 $\pm$ 300.8	305.7 $\pm$ 103.1	465.0 $\pm$ 36.7	309.4 $\pm$ 22.4	–	24.0 $\pm$ 11.6	2.09 $\pm$ 0.38
COS-PLAY w/ SHARED BANK	128.50 $\pm$ 133.12	63.31 $\pm$ 20.71	368.00 $\pm$ 42.69	937.94 $\pm$ 129.91	–	26.0 $\pm$ 12.2	2.71 $\pm$ 0.19
COS-PLAY (BASE QWEN3-8B)	1589.0 $\pm$ 192.4	510.9 $\pm$ 199.5	648.8 $\pm$ 38.8	948.9 $\pm$ 153.2	924.4	39.0 $\pm$ 9.4	2.96 $\pm$ 0.20

Table 1: **Performance across game categories.** We report **reward** for 2048, Tetris, Candy Crush, and Super Mario Bros, **overall win rate** for Avalon, and **overall mean supply centers** for Diplomacy. All results are reported with **95% confidence intervals**, based on **16 evaluation rollouts** for single-player games and **10 rollouts per player** for multi-player games. **Conclusion:** COS-PLAY achieves a significant **25.1%** average improvement over GPT-5.4 on single-player games, while remaining competitive with SOTA LLMs on multi-player social reasoning tasks.

Game	#Skills	#Cats	Avg. Clauses	Tot. Sub-Ep	Max Inst	Avg. Inst	Gini	Avg. Ver
2048	13	6	0.92	593	236	45.6	0.718	3.1
TETRIS	6	2	4.67	295	190	49.2	0.634	4.2
CANDYCRUSH	6	4	13.67	203	78	33.8	0.498	2.0
SUPER MARIO BROS	20	9	7.55	281	74	14.1	0.543	2.5
AVALON	16	3	4.40	479	125	29.9	0.564	6.4
DIPLOMACY	64	5	6.52	814	45	12.7	0.524	2.4

Table 2: **Cross-run skill reusability across games.** We report the number of discovered skills (#Skills), number of categories (#Cats), average number of clauses per skill (Avg. Clauses), total number of segmented sub-episodes (Tot. Sub-Ep), the maximum number of instances for the most reused skill (Max Inst), the average number of instances per skill (Avg. Inst), reuse concentration (Gini), and the average number of contract refinement versions per skill (Avg. Ver). These results suggest that our framework learns reusable and adaptable skill abstractions rather than memorizing isolated trajectories.

skills from unlabeled rollouts for future decisions. Our framework improves both sides jointly: the decision agent learns better skill retrieval and action generation, while the skill-bank agent continually extracts, refines, and updates skills and their contracts. By keeping the policy and skill bank aligned over time, COS-PLAY improves long-horizon control and adaptation. Experiments across diverse game environments show that COS-PLAY yields strong gains over frontier LLM baselines on single-player games while remaining competitive on multi-player social reasoning tasks.

Limitation and Future Work. A main limitation of COS-PLAY is its reliance on compact text-based state summaries, which limits grounded multi-modal understanding and can miss critical evidence in raw visual observations or long temporal dependencies. Over long trajectories, sum-

marization errors can accumulate, reducing skill relevance. An important direction for future work is to extend COS-PLAY to multi-modal interactive environments, where learned skills can capture both interaction dynamics and visual reasoning. We also aim to improve cross-domain transfer, so that learned skills generalize across multi-modal games, agentic environments, and broader visual reasoning tasks.

Limitations

A main limitation of COS-PLAY is its reliance on compact text-based state summaries, which limits its grounded multi-modal understanding and can miss critical evidence in raw visual observations or long temporal dependencies. Over long trajectories, summarization errors can accumulate, reducing skill relevance. Extending COS-PLAY to multi-modal interactive environments, where learned skills capture both interaction dynamics and visual reasoning, and improving cross-domain skill transfer remain important directions for future work.

References

Anthropic. 2026. Introducing claude sonnet 4.6.

Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, and 1 others. 2019. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*.

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. Openai gym. *arXiv preprint arXiv:1606.01540*.

Peng Chen, Pi Bu, Jun Song, Yuan Gao, and Bo Zheng. 2024. Can vlms play action role-playing games? take black myth wukong as a study case. *arXiv preprint arXiv:2409.12889*.

Alexander Duffy, Samuel J Paech, Ishana Shastri, Elizabeth Karpinski, Baptiste Alloui-Cros, Tyler Marques, and Matthew Lyle Olson. 2026. Democratizing diplomacy: A harness for evaluating any large language model on full-press diplomacy. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 37350–37359.

Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. 2025. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*.

Google DeepMind. 2026. Gemini 3.1 pro model card.

Yicheng He, Chengsong Huang, Zongxia Li, Jiaxin Huang, and Yonghui Yang. 2025. Visplay: Self-evolving vision-language models from images. *arXiv preprint arXiv:2511.15661*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, and 1 others. 2022. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3.

Hengyuan Hu and Dorsa Sadigh. 2023. Language instructed reinforcement learning for human-ai coordination. In *International Conference on Machine Learning*, pages 13584–13598. PMLR.

Lanxiang Hu, Mingjia Huo, Yuxuan Zhang, Haoyang Yu, Eric P Xing, Ion Stoica, Tajana Rosing, Haojian Jin, and Hao Zhang. 2025. Igame-bench: How good are llms at playing games? *arXiv preprint arXiv:2505.15146*.

Sihao Hu, Tiansheng Huang, Gaowen Liu, Ramana Rao Kompella, Fatih Ilhan, Selim Furkan Tekin, Yichang Xu, Zachary Yahn, and Ling Liu. 2024. A survey on large language model-based game agents. *arXiv preprint arXiv:2404.02039*.

Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. 2025a. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*.

Xu Huang, Junwu Chen, Yuxing Fei, Zhuohan Li, Philippe Schwallier, and Gerbrand Ceder. 2025b. Cascade: Cumulative agentic skill creation through autonomous development and evolution. *arXiv preprint arXiv:2512.23880*.

Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and 1 others. 2026a. Xskill: Continual learning from experience and skills in multimodal agents. *arXiv preprint arXiv:2603.12056*.

Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. 2026b. Sok: Agentic skills-beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*.

George Konidaris, Scott Kuindersma, Roderic Grupen, and Andrew Barto. 2010. Constructing skill trees for reinforcement learning agents from demonstration trajectories. *Advances in neural information processing systems*, 23.

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, and 1 others. 2026a. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.

Zaijing Li, Yuquan Xie, Rui Shao, Gongwei Chen, Dongmei Jiang, and Liqiang Nie. 2024. Optimus-1: Hybrid multimodal memory empowered agents excel in long-horizon tasks. *Advances in neural information processing systems*, 37:49881–49913.

Zongxia Li, Hongyang Du, Chengsong Huang, Xiyang Wu, Lantao Yu, Yicheng He, Jing Xie, Xiaomin Wu, Zhichao Liu, Jiarui Zhang, and 1 others. 2026b.

764	Mm-zero: Self-evolving multi-model vision lan-	Dongmin Park, Minkyu Kim, Beongjun Choi, Junhyuck	818
765	guage models from zero data. <i>arXiv preprint</i>	Kim, Keon Lee, Jonghyun Lee, Inkyu Park, Byeong-	819
766	<i>arXiv:2603.09206</i> .	Uk Lee, Jaeyoung Hwang, Jaewoo Ahn, and 1 others.	820
767	Zongxia Li, Wenhao Yu, Chengsong Huang, Rui Liu,	2025. Orak: A foundational benchmark for training	821
768	Zhenwen Liang, Fuxiao Liu, Jingxi Che, Dian Yu,	and evaluating llm agents on diverse video games.	822
769	Jordan Boyd-Graber, Haitao Mi, and 1 others. 2025.	<i>arXiv preprint arXiv:2506.03610</i> .	823
770	Self-rewarding vision-language model via reasoning	Siyuan Qi and Song-Chun Zhu. 2018. Intent-aware	824
771	decomposition. <i>arXiv preprint arXiv:2508.19652</i> .	multi-agent reinforcement learning. In <i>2018 IEEE</i>	825
772	Yi Liao, Yu Gu, Yuan Sui, Zining Zhu, Yifan Lu, Guo-	<i>international conference on robotics and automation</i>	826
773	hua Tang, Zhongqian Sun, and Wei Yang. 2025.	(ICRA), pages 7533–7540. IEEE.	827
774	Think in games: Learning to reason in games via	Maria Abi Raad, Arun Ahuja, Catarina Barros, Fred-	828
775	reinforcement learning with large language models.	eric Besse, Andrew Bolt, Adrian Bolton, Bethanie	829
776	<i>arXiv preprint arXiv:2508.21365</i> .	Brownfield, Gavin Buttimore, Max Cant, Sarah	830
777	Jonathan Light, Min Cai, Sheng Shen, and Ziniu Hu.	Chakera, and 1 others. 2024. Scaling instructable	831
778	2023. Avalonbench: Evaluating llms playing the	agents across many simulated worlds. <i>arXiv preprint</i>	832
779	game of avalon. <i>arXiv preprint arXiv:2310.05036</i> .	<i>arXiv:2404.10179</i> .	833
780	Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri	Shahab Rahimirad, Guven Gergerli, Lucia Romero, An-	834
781	Edwards, Bowen Baker, Teddy Lee, Jan Leike,	gela Qian, Matthew Lyle Olson, Simon Stepputtis,	835
782	John Schulman, Ilya Sutskever, and Karl Cobbe.	and Joseph Campbell. 2025. Bayesian social deduc-	836
783	2023. Let’s verify step by step. <i>arXiv preprint</i>	tion with graph-informed language models. <i>arXiv</i>	837
784	<i>arXiv:2305.20050</i> .	<i>preprint arXiv:2506.17788</i> .	838
785	Xiaoqian Liu, Ke Wang, Yuchuan Wu, Fei Huang, Yong-	Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu,	839
786	bin Li, Junge Zhang, and Jianbin Jiao. 2025. Agentic	Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan	840
787	reinforcement learning with implicit step rewards.	Zhang, YK Li, Yang Wu, and 1 others. 2024.	841
788	<i>arXiv preprint arXiv:2509.19199</i> .	Deepseekmath: Pushing the limits of mathematical	842
789	Wenxuan Lu, Jiangyang He, Zhanqiu Zhang, Steven Y	reasoning in open language models. <i>arXiv preprint</i>	843
790	Guo, and Tianning Zang. 2025. Cultivating gaming	<i>arXiv:2402.03300</i> .	844
791	sense for yourself: Making vlms gaming experts.	Tianye Shu, Jialin Liu, and Georgios N Yannakakis.	845
792	In <i>Proceedings of the 63rd Annual Meeting of the</i>	2021. Experience-driven pcg via reinforcement learn-	846
793	<i>Association for Computational Linguistics (Volume</i>	ing: A super mario bros study. In <i>2021 IEEE Confer-</i>	847
794	<i>1: Long Papers)</i> , pages 13132–13152.	<i>ence on Games (CoG)</i> , pages 1–9. IEEE.	848
795	Weiyu Ma, Yuqian Fu, Zecheng Zhang, Guohao Li, and	David Silver, Thomas Hubert, Julian Schrittwieser, Ioan-	849
796	Bernard Ghanem. 2025. Vlms play starcraft ii: A	nis Antonoglou, Matthew Lai, Arthur Guez, Marc	850
797	benchmark and multimodal decision method. <i>arXiv</i>	Lancot, Laurent Sifre, Dharshan Kumaran, Thore	851
798	<i>preprint arXiv:2503.05383</i> .	Graepel, and 1 others. 2017. Mastering chess and	852
799	Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li,	shogi by self-play with a general reinforcement learn-	853
800	Yisen Wang, Haifeng Zhang, and Jun Wang. 2026.	ing algorithm. <i>arXiv preprint arXiv:1712.01815</i> .	854
801	Procmem: Learning reusable procedural memory	Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Man-	855
802	from experience via non-parametric ppo for llm	dlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and	856
803	agents. <i>arXiv preprint arXiv:2602.01869</i> .	Anima Anandkumar. 2023. Voyager: An open-ended	857
804	Volodymyr Mnih, Koray Kavukcuoglu, David Sil-	embodied agent with large language models. <i>arXiv</i>	858
805	ver, Alex Graves, Ioannis Antonoglou, Daan Wier-	<i>preprint arXiv:2305.16291</i> .	859
806	stra, and Martin Riedmiller. 2013. Playing atari	Jiong Xiao Wang, Qiaojing Yan, Yawei Wang, Yijun	860
807	with deep reinforcement learning. <i>arXiv preprint</i>	Tian, Soumya Smruti Mishra, Zhichao Xu, Megha	861
808	<i>arXiv:1312.5602</i> .	Gandhi, Panpan Xu, and Lin Lee Cheong. 2025a.	862
809	OpenAI. 2025. Introducing gpt-oss.	Reinforcement learning for self-improving agent with	863
810	OpenAI. 2026a. GPT-5 mini Model (gpt-5-mini).	skill library. <i>arXiv preprint arXiv:2512.17102</i> .	864
811	OpenAI. 2026b. Introducing gpt-5.4.	Qinsi Wang, Bo Liu, Tianyi Zhou, Jing Shi, Yueqian	865
812	Davide Paglieri, Bartłomiej Cupiał, Samuel Coward,	Lin, Yiran Chen, Hai Helen Li, Kun Wan, and Wen-	866
813	Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Ed-	tian Zhao. 2025b. Vision-zero: Scalable vlm self-	867
814	uardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob	improvement via strategic gamified self-play. <i>arXiv</i>	868
815	Fergus, and 1 others. 2024. Balrog: Benchmark-	<i>preprint arXiv:2509.25541</i> .	869
816	ing agentic llm and vlm reasoning on games. <i>arXiv</i>	Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni,	870
817	<i>preprint arXiv:2411.13543</i> .	Abhranil Chandra, Shiguang Guo, Weiming Ren,	871
		Aaran Arulraj, Xuan He, Ziyang Jiang, and 1 others.	872
		2024. Mmlu-pro: A more robust and challenging	873

874	multi-task language understanding benchmark. <i>Advances in Neural Information Processing Systems</i> , 37:95266–95290.	Simon Yu, Gang Li, Weiyang Shi, and Peng Qi. 2025b. Polyskill: Learning generalizable skills through polymorphic abstraction. <i>arXiv preprint arXiv:2510.15863</i> .	928
875			929
876			930
877	Zirui Wang, Junyi Zhang, Jiaxin Ge, Long Lian, Letian Fu, Lisa Dunlap, Ken Goldberg, XuDong Wang, Ion Stoica, David M Chan, and 1 others. 2026. Visgym: Diverse, customizable, scalable environments for multimodal agents. <i>arXiv preprint arXiv:2601.16973</i> .		931
878			932
879			933
880			934
881			935
882			936
883	Xiyang Wu, Rohan Chandra, Tianrui Guan, Amrit Bedi, and Dinesh Manocha. 2023. Intent-aware planning in heterogeneous traffic via distributed multi-agent reinforcement learning. In <i>Conference on Robot Learning</i> , pages 446–477. PMLR.	Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, and 1 others. 2025. Agentevolver: Towards efficient self-evolving agent system. <i>arXiv preprint arXiv:2511.10395</i> .	937
884			938
885			939
886			940
887			941
888	Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, and 1 others. 2026. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. <i>arXiv preprint arXiv:2602.08234</i> .	Alex L Zhang, Thomas L Griffiths, Karthik R Narasimhan, and Ofir Press. 2025. Videogamebench: Can vision-language models complete popular video games? <i>arXiv preprint arXiv:2505.18134</i> .	942
889			943
890			944
891			945
892			
893			946
894	Zhishang Xiang, Chengyi Yang, Zerui Chen, Zhimin Wei, Yunbo Tang, Zongpei Teng, Zexi Peng, Zongxia Li, Chengsong Huang, Yicheng He, and 1 others. 2025. A systematic survey of self-evolving agents: From model-centric to environment-driven co-evolution. <i>Researchgate</i> .	Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026a. Memskill: Learning and evolving memory skills for self-evolving agents. <i>arXiv preprint arXiv:2602.02474</i> .	947
895			948
896			949
897			950
898			
899			951
900	Han Xiao, Guozhi Wang, Hao Wang, Shilong Liu, Yuxiang Chai, Yue Pan, Yufeng Zhou, Xiaoxin Chen, Yafei Wen, and Hongsheng Li. 2026. Ui-mem: Self-evolving experience memory for online reinforcement learning in mobile gui agents. <i>arXiv preprint arXiv:2602.05832</i> .	Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, and 1 others. 2026b. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. <i>arXiv preprint arXiv:2601.03192</i> .	952
901			953
902			954
903			955
904			956
905			
906	Renjun Xu and Yang Yan. 2026. Agent skills for large language models: Architecture, acquisition, security, and the path forward. <i>arXiv preprint arXiv:2602.12430</i> .		
907			
908			
909			
910	Zelai Xu, Zhexuan Xu, Xiangmin Yi, Huining Yuan, Xinlei Chen, Yi Wu, Chao Yu, and Yu Wang. 2025a. Vs-bench: Evaluating vlms for strategic reasoning and decision-making in multi-agent environments.		
911			
912			
913			
914	Zhongwen Xu, Xianliang Wang, Siyi Li, Tao Yu, Liang Wang, Qiang Fu, and Wei Yang. 2025b. Agents play thousands of 3d video games. <i>arXiv preprint arXiv:2503.13356</i> .		
915			
916			
917			
918	An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. <i>arXiv preprint arXiv:2505.09388</i> .		
919			
920			
921			
922			
923	Shuo Yu, Mingyue Cheng, Daoyu Wang, Qi Liu, Zirui Liu, Ze Guo, and Xiaoyu Tao. 2025a. Memweaver: A hierarchical memory from textual interactive behaviors for personalized generation. <i>arXiv preprint arXiv:2510.07713</i> .		
924			
925			
926			
927			

A Reproducibility

We will release the full COS-PLAY implementation, including the decision agent, skill bank agent, co-evolution training loop, GRPO training scripts with multi-adapter LoRA configurations, skill bank pipeline, and all prompt templates used in this paper. We will also provide the environment wrappers, reward definitions, and configuration files needed to reproduce the experiments across all six game environments.

We document the full evaluation protocol in the paper and appendix, including environment settings (Appendix B), hyperparameters, the number of evaluation rollouts per setting (Appendix C), and reward definitions (Appendix G). For single-player games, we report results over 16 rollouts; for multi-player games, over 10 rollouts per player. Because multi-player experiments use black-box LLM APIs as opponents (GPT-5-mini for training, GPT-5.4 for evaluation), exact numerical replication may vary across API versions and infrastructure conditions. To reduce this variance, we use fixed prompts, deterministic environment seeds where supported, and repeated runs under the same protocol. We therefore expect the main qualitative conclusions to remain reproducible even when exact per-run metrics vary slightly.

Cost Disclosure. We disclose the main computational costs of COS-PLAY training in Table 3. Across six games, the SFT teacher stage required 119,308 GPT-5.4 calls, corresponding to \$510.54 in measured API cost. For multiplayer training, we used GPT-5-mini opponents to reduce rollout cost, adding 125,619 calls and \$30.36 in API cost across Avalon and Diplomacy. The scaled COS-PLAY training budget used 412.1 GPU-hours in total, decomposed into 173.0 GPU-hours for decision-agent training and 239.1 GPU-hours for skill-bank training and curation. This cost includes teacher trajectories, environment rollouts, GRPO, and multiple LoRA adapters.

B Game Environment Settings

We evaluate on six game environments spanning puzzle solving, platform control, and multi-agent social reasoning. In all cases, observations are converted into textual or natural-language state descriptions, and the agent interacts through discrete text actions. Below we summarize the observation space, action space, horizon, and reward for each environment.

2048. 2048 is a single-player sliding-tile puzzle on a 4×4 grid. Equal-valued tiles merge on contact, and the game ends when no valid moves remain. The agent observes a text-rendered board with tile values, score, maximum tile, empty-cell count, and directional move lookahead. The action space has four actions: up, down, left, and right. Episodes are capped at 200 steps, with reward given by the merge score.

Candy Crush. Candy Crush is a single-player match-3 puzzle on an 8×8 board with four candy colors. The player swaps adjacent candies to form matches, after which matched candies are cleared and new candies fall. The agent observes a text-based board and the currently valid swap actions. The action space is dynamic and consists of valid coordinate-pair swaps at each step. Episodes last at most 50 moves, and rewards are based on points from matches and cascades.

Tetris. Tetris is a single-player tile-stacking game on a 10×20 board with the standard seven tetrominoes and a four-piece preview queue. The agent receives an ASCII board, the current piece, upcoming pieces, and summary statistics such as stack height, holes, cleared lines, and level. Instead of primitive controls, the environment provides macro actions corresponding to valid placements defined by rotation and target column. Episodes are capped at 200 steps, and reward is the game score.

Super Mario Bros. Super Mario Bros. is a single-player side-scrolling platform game where Mario must traverse the level, avoid hazards, and reach the flag. Observations are converted into natural-language descriptions of Mario’s position, nearby objects and enemies, and relevant game-state information. The action space contains seven discrete actions: noop, right, right+A, A, left, left+A, and down. Episodes are capped at 200 steps. Rewards combine progress and in-game signals, including distance, coins, and time bonus.

Avalon. Avalon is a five-player social deduction game with hidden roles: Merlin, two Servants, one Minion, and one Assassin. Play proceeds through quest rounds with team proposal, voting, quest execution, and assassination phases. The agent observes a natural-language state description containing the phase, private role information, quest progress, leader identity, proposed team, discussion history, and vote outcomes. Actions depend on the phase and include team proposals, votes,

Game	Env. Steps	GPT-5.4 Teacher Calls	Teacher API Cost	GPT-5-mini Calls	GPT-5-mini Cost	Decision Agent GPU-h	Skill-Bank Agent GPU-h	Total GPU-h
2048	8,125	47,714	\$155.96	–	–	13.6	65.2	78.8
Candy Crush	3,000	18,013	\$61.63	–	–	8.7	25.7	34.4
Tetris	3,700	22,222	\$92.03	–	–	21.1	64.7	85.8
Super Mario	3,043	18,274	\$66.04	–	–	90.5	58.6	149.1
Avalon	976	5,869	\$32.76	32,213	\$3.74	7.3	17.9	25.2
Diplomacy	1,200	7,216	\$102.12	93,406	\$26.62	31.8	7.0	38.8
Total	20,044	119,308	\$510.54	125,619	\$30.36	173.0 (42.0%)	239.1 (58.0%)	412.1

Table 3: **Cost-normalized training budget across six games.** We report one-time GPT-5.4 teacher calls, GPT-5-mini opponent calls for multiplayer training, and GPU-hours for COS-PLAY training. Teacher calls cover cold-start rollouts, intent and skill-aware labeling, and skill-bank extraction. GPT-5-mini opponents are used only for Avalon and Diplomacy. In total, COS-PLAY uses 412.1 GPU-hours: 173.0 for the decision agent and 239.1 for the skill-bank agent.

quest choices, and assassination targets. Each game contains up to five quests, and rewards are based mainly on whether the agent’s side wins.

Diplomacy. Diplomacy is a seven-player grand-strategy board game on the standard European map. The powers are Austria, England, France, Germany, Italy, Russia, and Turkey, and the game cycles through movement, retreat, and adjustment phases. The agent observes a natural-language state including the current phase, controlled power, unit locations, supply center counts, valid orders, recent negotiations, and phase history. Actions are standard Diplomacy order strings issued jointly for all controlled units in a phase. Episodes are capped at 20 phases, and rewards are based on the number of controlled supply centers normalized by the victory target of 18.

C Key Hyperparameters

We summarize the main hyperparameters used in co-evolution training for the six game environments in the main paper. Table 4 lists the game-specific settings used in our main experiments. All training runs are conducted on an 8×A100 GPU cluster. For games without explicit GRPO overrides, we report the default values directly: GRPO clip ratio 0.2, maximum 4 epochs, no advantage clipping, learning rate 5×10^{-5} , and KL coefficient 0.05.

D Breakdown of Role-wise Performance on Multi-player Games

For multi-player social games, we use GPT-5.4 as the opponent model. We report the main results in Table 1, with role-wise and power-wise breakdowns provided in Table 5 and Table 6.

Avalon is a team-based competitive game in which only one side can win. It is structurally harder for the Good side, since Good players must infer hidden roles from sparse signals such as proposals, votes, and quest outcomes, while Evil players begin with full coordination and can strategically hide or sabotage (Light et al., 2023). As shown in Table 1, our method achieves performance comparable to GEMINI-3.1-PRO and GPT-OSS-120B, with a slight 1% lower in win rate. This setting highlights a key weakness of baseline LLMs: they often struggle to maintain consistent beliefs, track intent across rounds, and act coherently from prior evidence. Our method alleviates this by converting raw histories into structured states and applying phase-specific skills, enabling more consistent suspicion modeling and multi-round commitment.

Diplomacy further amplifies these challenges, as it requires long-context reasoning, negotiation, alliance tracking, and multi-phase planning. While baselines often fall back to defensive, non-cooperative behavior against strong opponents, our method benefits from iterative skill-bank refinement, which provides reusable strategic patterns and improves few-shot adaptation over repeated interactions, inducing at least 8.8% better performance over GEMINI-3.1-PRO. Overall, social games expose the weakness of stateless LLM agents, whereas our framework supports long-horizon social reasoning through persistent

Game	Total Steps	Episodes / Step	Ckpt Int.	GRPO LR	KL Coeff.	Clip	Max Epochs	Adv. Clip
2048	10	8	3	5×10^{-5}	0.05	0.20	4	–
Candy Crush	10	8	3	5×10^{-5}	0.05	0.20	4	–
Tetris	7	8	1	2×10^{-5}	0.08	0.10	2	3.0
Super Mario Bros.	20	8	1	3×10^{-5}	0.04	0.15	3	5.0
Avalon	20	20	1	2×10^{-5}	0.06	0.15	2	3.0
Diplomacy	25	28	1	1×10^{-5}	0.08	0.12	2	3.0

Table 4: Key co-evolution training hyperparameters across the six game environments in the main paper. “Ckpt Int.” denotes checkpoint interval. For 2048 and Candy Crush, all GRPO fields use the default values.

Model	Avalon						Overall
	Good			Evil			Avg. Win Rate $\uparrow$
	Merlin Win Rate $\uparrow$	Servant Win Rate $\uparrow$	Good Avg. Win Rate $\uparrow$	Assassin Win Rate $\uparrow$	Minion Win Rate $\uparrow$	Evil Avg. Win Rate $\uparrow$	
GPT-5.4	6.2 ± 14.0	6.2 ± 22.1	6.2 ± 10.6	93.8 ± 22.1	93.8 ± 22.1	93.8 ± 18.4	41.2 ± 10.6
GEMINI-3.1-PRO	10.0 ± 19.3	36.4 ± 18.7	28.1 ± 14.9	66.7 ± 30.2	66.7 ± 23.6	66.7 ± 20.0	42.0 ± 13.2
CLAUDE-4.6	30.0 ± 24.8	40.9 ± 19.0	37.5 ± 15.9	33.3 ± 30.2	50.0 ± 24.6	44.4 ± 20.9	40.0 ± 13.1
GPT-OSS-120B	30.0 ± 24.8	31.8 ± 18.2	31.2 ± 15.3	50.0 ± 31.2	58.3 ± 24.4	55.6 ± 20.9	40.0 ± 13.1
QWEN3-8B	18.8 ± 18.2	18.4 ± 12.1	18.5 ± 10.2	66.7 ± 23.6	42.9 ± 23.0	53.8 ± 17.9	30.0 ± 9.9
SFT w/o SKILL	18.8 ± 18.2	18.4 ± 12.1	18.5 ± 10.2	41.7 ± 24.4	57.1 ± 23.0	50.0 ± 17.9	28.7 ± 9.7
SFT + 1ST SKILL	6.2 ± 13.6	13.2 ± 10.8	11.1 ± 8.5	33.3 ± 23.6	50.0 ± 23.2	42.3 ± 17.8	21.2 ± 8.9
SFT + FINAL SKILL	25.0 ± 19.7	26.3 ± 13.5	25.9 ± 11.4	8.3 ± 17.0	35.7 ± 22.4	23.1 ± 15.5	25.0 ± 9.3
GRPO w/o SKILL	37.5 ± 21.4	15.8 ± 11.5	22.2 ± 10.9	58.3 ± 24.4	71.4 ± 21.5	65.4 ± 17.2	36.2 ± 10.3
GRPO + 1ST SKILL	31.2 ± 20.7	18.4 ± 12.1	22.2 ± 10.9	41.7 ± 24.4	57.1 ± 23.0	50.0 ± 17.9	31.2 ± 10.0
GRPO NO-BANK	0.0 ± 27.8	13.6 ± 18.6	9.4 ± 9.9	33.3 ± 36.7	41.7 ± 24.3	38.9 ± 21.8	20.0 ± 10.9
GRPO FIXED-BANK	10.0 ± 38.6	22.7 ± 20.6	18.8 ± 12.7	33.3 ± 36.7	25.0 ± 28.2	27.8 ± 21.6	22.0 ± 11.2
GPT-5.4 + FINAL SKILL	6.2 ± 11.9	6.2 ± 8.4	6.2 ± 6.8	93.8 ± 11.9	93.8 ± 11.9	93.8 ± 8.4	41.2 ± 10.8
MEMSKILL	30.0 ± 24.8	26.3 ± 18.5	27.6 ± 15.5	22.2 ± 24.2	25.0 ± 22.2	23.8 ± 17.2	26.0 ± 11.8
MEMRL	50.0 ± 26.3	36.8 ± 19.9	41.4 ± 16.9	20.0 ± 22.7	36.4 ± 24.7	28.6 ± 18.1	36.0 ± 12.9
SKILLRL	20.0 ± 22.7	26.3 ± 18.5	24.1 ± 14.9	11.1 ± 20.8	33.3 ± 23.6	23.8 ± 17.2	24.0 ± 11.6
COS-PLAY w/ SHARED BANK	0.0 ± 0.0	0.0 ± 0.0	66.7 ± 37.7	75.0 ± 24.5	0.0 ± 0.0	72.2 ± 20.7	26.0 ± 12.2
COS-PLAY (BASE QWEN3-8B)	33.3 ± 17.7	23.3 ± 12.2	26.9 ± 10.4	64.3 ± 22.5	63.2 ± 20.0	63.6 ± 15.6	39.0 ± 9.4

Table 5: **Per-role win rates for Avalon.** We report win rates with 95% confidence for both good-side roles (Merlin, Servant), evil-side roles (Assassin, Minion). Results show that our method reaches comparable performance to GEMINI-3.1-PRO and GPT-OSS-120B.

state tracking, cross-turn memory, and skill-guided decision-making.

E Multi-play Game Ablations

F Necessity of Using Different LoRA

In our framework, both the decision agent and the skill-bank agent use multiple LoRA adapters for different functional stages. Specifically, the decision agent uses two LoRAs for skill retrieval and action generation, while the skill-bank agent uses three LoRAs for trajectory segmentation, contract generation, and skill-bank maintenance. To better understand the role of each adapter and the effect of splitting LoRAs by function, we conduct a supplementary experiment on CANDY CRUSH in Table 9, comparing COS-PLAY with stage-specific LoRAs against a merged variant that uses only one LoRA for the decision agent and one for the skill-bank agent. The results show that the merged-LoRA variant performs worse during training, suggesting that splitting LoRAs is important for co-evolution.

Model	Diplomacy							Overall
	Austria Mean SC $\uparrow$	England Mean SC $\uparrow$	France Mean SC $\uparrow$	Germany Mean SC $\uparrow$	Italy Mean SC $\uparrow$	Russia Mean SC $\uparrow$	Turkey Mean SC $\uparrow$	Mean SC $\uparrow$
GPT-5.4	4.38 $\pm$ 1.34	4.12 $\pm$ 0.82	4.50 $\pm$ 1.09	5.12 $\pm$ 0.95	4.50 $\pm$ 0.77	5.12 $\pm$ 1.52	5.12 $\pm$ 0.95	4.70 $\pm$ 0.35
GEMINI-3.1-PRO	2.50 $\pm$ 0.89	2.38 $\pm$ 0.63	3.12 $\pm$ 0.70	2.88 $\pm$ 0.82	3.14 $\pm$ 0.35	3.14 $\pm$ 0.35	1.86 $\pm$ 1.24	2.72 $\pm$ 0.26
CLAUDE-4.6	3.20 $\pm$ 0.56	2.90 $\pm$ 0.41	3.50 $\pm$ 0.70	3.78 $\pm$ 0.64	3.20 $\pm$ 0.30	2.80 $\pm$ 0.30	2.80 $\pm$ 0.66	3.16 $\pm$ 0.19
GPT-OSS-120B	1.75 $\pm$ 1.07	2.88 $\pm$ 0.29	2.88 $\pm$ 0.29	2.62 $\pm$ 0.44	3.00 $\pm$ 0.00	2.88 $\pm$ 0.29	1.25 $\pm$ 0.97	2.46 $\pm$ 0.25
QWEN3-8B	1.62 $\pm$ 0.26	2.75 $\pm$ 0.25	2.75 $\pm$ 0.16	2.88 $\pm$ 0.12	3.00 $\pm$ 0.00	2.88 $\pm$ 0.23	2.62 $\pm$ 0.18	2.64 $\pm$ 0.18
SFT w/o SKILL	2.12 $\pm$ 0.23	2.75 $\pm$ 0.16	2.88 $\pm$ 0.12	2.62 $\pm$ 0.26	2.88 $\pm$ 0.12	3.00 $\pm$ 0.00	3.00 $\pm$ 0.00	2.75 $\pm$ 0.13
SFT + 1ST SKILL	2.12 $\pm$ 0.30	2.88 $\pm$ 0.12	3.12 $\pm$ 0.30	2.50 $\pm$ 0.33	3.12 $\pm$ 0.12	3.25 $\pm$ 0.25	3.25 $\pm$ 0.16	2.89 $\pm$ 0.20
SFT + FINAL SKILL	2.12 $\pm$ 0.44	2.75 $\pm$ 0.16	2.25 $\pm$ 0.25	2.71 $\pm$ 0.29	2.88 $\pm$ 0.23	2.88 $\pm$ 0.12	3.00 $\pm$ 0.57	2.65 $\pm$ 0.25
GRPO w/o SKILL	2.33 $\pm$ 0.33	2.83 $\pm$ 0.17	3.00 $\pm$ 0.00	2.80 $\pm$ 0.20	3.00 $\pm$ 0.00	3.20 $\pm$ 0.20	2.20 $\pm$ 0.37	2.76 $\pm$ 0.19
GRPO + 1ST SKILL	1.38 $\pm$ 0.18	3.12 $\pm$ 0.12	2.25 $\pm$ 0.16	2.75 $\pm$ 0.37	3.00 $\pm$ 0.00	3.25 $\pm$ 0.16	2.14 $\pm$ 0.26	2.56 $\pm$ 0.22
GRPO NO-BANK	1.80 $\pm$ 1.00	2.80 $\pm$ 0.30	2.90 $\pm$ 0.23	2.30 $\pm$ 0.48	3.00 $\pm$ 0.00	3.00 $\pm$ 0.34	2.50 $\pm$ 0.61	2.61 $\pm$ 0.19
GRPO FIXED-BANK	1.80 $\pm$ 0.66	2.90 $\pm$ 0.23	2.60 $\pm$ 0.37	2.20 $\pm$ 0.94	2.90 $\pm$ 0.23	3.00 $\pm$ 0.00	2.40 $\pm$ 0.60	2.54 $\pm$ 0.19
GPT-5.4 + FINAL SKILL	3.50 $\pm$ 0.65	3.94 $\pm$ 0.58	5.06 $\pm$ 0.52	4.75 $\pm$ 0.61	4.81 $\pm$ 0.63	6.81 $\pm$ 0.86	4.50 $\pm$ 0.51	4.77 $\pm$ 0.28
MEMSKILL	1.99 $\pm$ 0.29	2.96 $\pm$ 0.07	2.88 $\pm$ 0.16	0.41 $\pm$ 0.81	2.45 $\pm$ 0.30	2.97 $\pm$ 0.02	2.81 $\pm$ 0.29	2.35 $\pm$ 0.31
MEMRL	2.10 $\pm$ 0.41	3.00 $\pm$ 0.00	2.60 $\pm$ 0.50	2.30 $\pm$ 0.83	3.00 $\pm$ 0.34	3.40 $\pm$ 0.37	2.90 $\pm$ 0.23	2.76 $\pm$ 0.17
SKILLRL	0.41 $\pm$ 0.55	3.01 $\pm$ 0.00	2.41 $\pm$ 1.18	1.63 $\pm$ 0.81	2.55 $\pm$ 0.64	2.55 $\pm$ 0.62	2.10 $\pm$ 1.09	2.09 $\pm$ 0.38
COS-PLAY w/ SHARED BANK	2.40 $\pm$ 0.43	3.00 $\pm$ 0.00	2.40 $\pm$ 0.32	2.90 $\pm$ 0.20	3.00 $\pm$ 0.00	3.40 $\pm$ 0.32	1.90 $\pm$ 0.62	2.71 $\pm$ 0.19
COS-PLAY (BASE QWEN3-8B)	2.22 $\pm$ 0.55	2.70 $\pm$ 0.42	3.10 $\pm$ 0.46	3.12 $\pm$ 0.58	3.20 $\pm$ 0.39	3.30 $\pm$ 0.30	3.00 $\pm$ 0.72	2.96 $\pm$ 0.20

Table 6: **Per-power mean supply centers (SC) for Diplomacy.** We report the mean supply centers achieved by each model for each power, together with the overall mean across all powers, with 95% confidence intervals. GPT-5.4 is evaluated in self-play, while all other models are evaluated against GPT-5.4. **Conclusion:** COS-PLAY outperforms GEMINI-3.1-PRO by 8.8% on average.

Model	Avalon						Overall
	Good			Evil			Avg. Win Rate $\uparrow$
	Merlin Win Rate $\uparrow$	Servant Win Rate $\uparrow$	Good Avg. Win Rate $\uparrow$	Assassin Win Rate $\uparrow$	Minion Win Rate $\uparrow$	Evil Avg. Win Rate $\uparrow$	
GPT-5.4 vs GPT-5-MINI	47.6 $\pm$ 19.6	50.0 $\pm$ 22.0	48.6 $\pm$ 15.5	66.7 $\pm$ 36.7	71.4 $\pm$ 35.5	69.2 $\pm$ 26.9	54.0 $\pm$ 13.6
vs GPT-5.4	6.2 $\pm$ 14.0	6.2 $\pm$ 22.1	6.2 $\pm$ 10.6	93.8 $\pm$ 22.1	93.8 $\pm$ 22.1	93.8 $\pm$ 18.4	41.2 $\pm$ 10.6
COS-PLAY vs GPT-5-MINI	88.9 $\pm$ 31.5	80.0 $\pm$ 42.4	85.7 $\pm$ 25.6	75.0 $\pm$ 44.9	50.0 $\pm$ 40.5	66.7 $\pm$ 36.7	80.0 $\pm$ 21.6
vs GPT-5.4	33.3 $\pm$ 17.7	23.3 $\pm$ 12.2	26.9 $\pm$ 10.4	64.3 $\pm$ 22.5	63.2 $\pm$ 20.0	63.6 $\pm$ 15.6	39.0 $\pm$ 9.4

Table 7: **Avalon role- and side-wise focal win rates.** We report win rates with 95% confidence intervals, separated by role and side to distinguish Good-side reasoning from Evil-side deception. Opponent strength strongly affects performance: GPT-5.4’s Good-side win rate drops from 48.6% against GPT-5-MINI to 6.2% in GPT-5.4 self-play, while COS-PLAY achieves the strongest Good-side result under matched GPT-5.4 opponents.

When diverse objectives are compressed into a single adapter, the learned capacity must be shared across incompatible functions, which can cause interference. As shown in Table 9, stage-specific LoRAs enable cleaner specialization, reduce optimization conflict, and better preserve functional capabilities over time.

G Reward Design

Both the decision agent and the skill bank agent are trained with GRPO using function-specific LoRA adapters. Each adapter has a dedicated reward function designed to reinforce the specific capability it governs. All reward computations are CPU-only and require no additional LLM inference.

G.1 Decision Agent Rewards

Action-Taking. The action-taking adapter is trained with a per-step reward

$$r_t = r_t^{\text{env}} + \lambda_f r_t^{\text{follow}} + r_t^{\text{cost}}, \quad (6)$$

where r_t^{env} is the environment reward, r_t^{follow} is a skill-following shaping term, and r_t^{cost} penalizes skill switching. The shaping term densifies sparse rewards during skill execution: satisfying a new predicate in the active skill’s effect contract gives a small bonus, satisfying all predicates gives a larger bonus, and steps with no progress receive a small penalty. The cost term applies a negative reward to skill switching, which is a common failure mode in early training. We set $\lambda_f = 0.1$ in all experiments.

Skill Retrieval. The skill-retrieval adapter is trained with a delayed reward evaluated at skill-

Model	Diplomacy							Overall
	Austria Mean SC $\uparrow$	England Mean SC $\uparrow$	France Mean SC $\uparrow$	Germany Mean SC $\uparrow$	Italy Mean SC $\uparrow$	Russia Mean SC $\uparrow$	Turkey Mean SC $\uparrow$	Mean SC $\uparrow$
COS-PLAY vs GPT-5-MINI	5.40 $\pm$ 3.01	4.29 $\pm$ 0.69	6.01 $\pm$ 1.97	5.21 $\pm$ 1.38	3.65 $\pm$ 1.02	5.76 $\pm$ 1.65	4.92 $\pm$ 1.94	5.04 $\pm$ 0.50
COS-PLAY vs GPT-5.4	2.22 $\pm$ 0.55	2.70 $\pm$ 0.42	3.10 $\pm$ 0.46	3.12 $\pm$ 0.58	3.20 $\pm$ 0.39	3.30 $\pm$ 0.30	3.00 $\pm$ 0.72	2.96 $\pm$ 0.20
GPT-5.4 vs GPT-5-MINI	5.43 $\pm$ 0.63	4.43 $\pm$ 0.37	4.57 $\pm$ 0.67	5.14 $\pm$ 0.84	3.86 $\pm$ 0.64	5.29 $\pm$ 1.03	5.29 $\pm$ 0.70	4.84 $\pm$ 0.25
GPT-5.4 SELF-PLAY	4.38 $\pm$ 1.34	4.12 $\pm$ 0.82	4.50 $\pm$ 1.09	5.12 $\pm$ 0.95	4.50 $\pm$ 0.77	5.12 $\pm$ 1.52	5.12 $\pm$ 0.95	4.70 $\pm$ 0.35

Table 8: **Diplomacy opponent-strength matrix.** We report focal mean supply centers (SC) for each power and the overall mean across powers, with 95% Student-*t* confidence intervals shown as mean $\pm$ half-width. The matrix compares focal agents under matched and shifted opponent settings. **Conclusion:** under GPT-5-MINI opponents, COS-PLAY 8B reaches 5.04 $\pm$ 0.50 mean SC, comparable to GPT-5.4’s 4.84 $\pm$ 0.25 mean SC under the same opponent. Under stronger GPT-5.4 opponents, the lower COS-PLAY scores reflect opponent-distribution shift rather than a failure of the skill bank.

Model	Reward $\uparrow$
GPT-5.4	532.6 $\pm$ 24.8
QWEN3-8B	519.9 $\pm$ 37.8
COS-PLAY (MERGED LoRAs)	502.5 $\pm$ 22.4
COS-PLAY (BASE QWEN3-8B)	648.8 $\pm$ 38.8

Table 9: **The necessity of splitting LoRAs.** We conduct a supplementary experiment on CANDY CRUSH, comparing COS-PLAY with stage-specific LoRAs against a merged variant that uses one LoRA for the decision agent and one for the skill-bank agent. The merged-LoRA variant performs worse during training, indicating that splitting LoRAs is important for handling the diverse objectives arising in co-evolution.

switch time. The reward is computed from the normalized environment reward accumulated during the skill, a temporal efficiency term, a contract-completion term, an abort penalty for violated preconditions, and a low-weight retrieval-confidence prior from the RAG module. This reward favors skills that are successful, efficient, and applicable in the current state.

G.2 Skill Bank Agent Rewards

Infer Segmentation. The segmentation adapter is trained to assign high-reward trajectory segments to reusable skills in the current bank. Its reward includes: (i) the fraction of positive episode reward covered by existing skills, (ii) a value-matching term between skill quality and segment importance, (iii) the Viterbi decode score for global consistency, and (iv) a decode-margin term for confident assignments. Assignments to *new skill* receive partial credit, preventing systematic overuse of existing skills when new behavior appears.

Contract Learning. The contract adapter is trained to produce compact effect contracts for each skill. Without held-out instances, the reward uses F1 between predicted effects and observed start/end predicate changes, with additional terms

for added/deleted predicate coverage, sparsity, and specificity. With held-out instances, the reward emphasizes generalization through a reward-weighted holdout pass rate, together with a precision/recall term against the consensus effects and a reward-alignment term favoring contracts that hold on high-value instances.

Skill Bank Curator. The curator adapter evaluates proposed bank updates, including materialize, merge, split, and retire operations. The reward includes: (i) a quality-alignment term from the continuous skill-quality score, (ii) an exploration term for approving promising new skills with adequate evidence, and (iii) a reason-quality term that favors evidence-based decisions using pass rates, quality scores, and instance counts. The exploration term helps prevent systematic rejection of new skills.

H Generalization onto General LLM Reasoning Tasks

We further evaluate on

Model	MMLU-Pro Acc. $\uparrow$	Math-500 EM $\uparrow$
QWEN3-8B	61.99%	46.40%
COS-PLAY	61.15%	44.60%

Table 10: **Overall performance on general reasoning benchmarks.** COS-PLAY remains comparable to the base model on both benchmarks.

on two standard LLM reasoning benchmarks, Math-500 (Hendrycks et al., 2021; Lightman et al., 2023) and MMLU-Pro (Wang et al., 2024), to assess whether our adaptation preserves the base model’s general reasoning ability. Math-500 measures mathematical reasoning, while MMLU-Pro evaluates broad knowledge and reasoning across diverse domains. As shown in Table 10, COS-PLAY achieves performance close to QWEN3-8B on both

1233 benchmarks, with absolute drops of only 0.8% on
1234 MMLU-Pro and 1.8% on Math-500, indicating that
1235 our method largely preserves general reasoning
1236 performance.

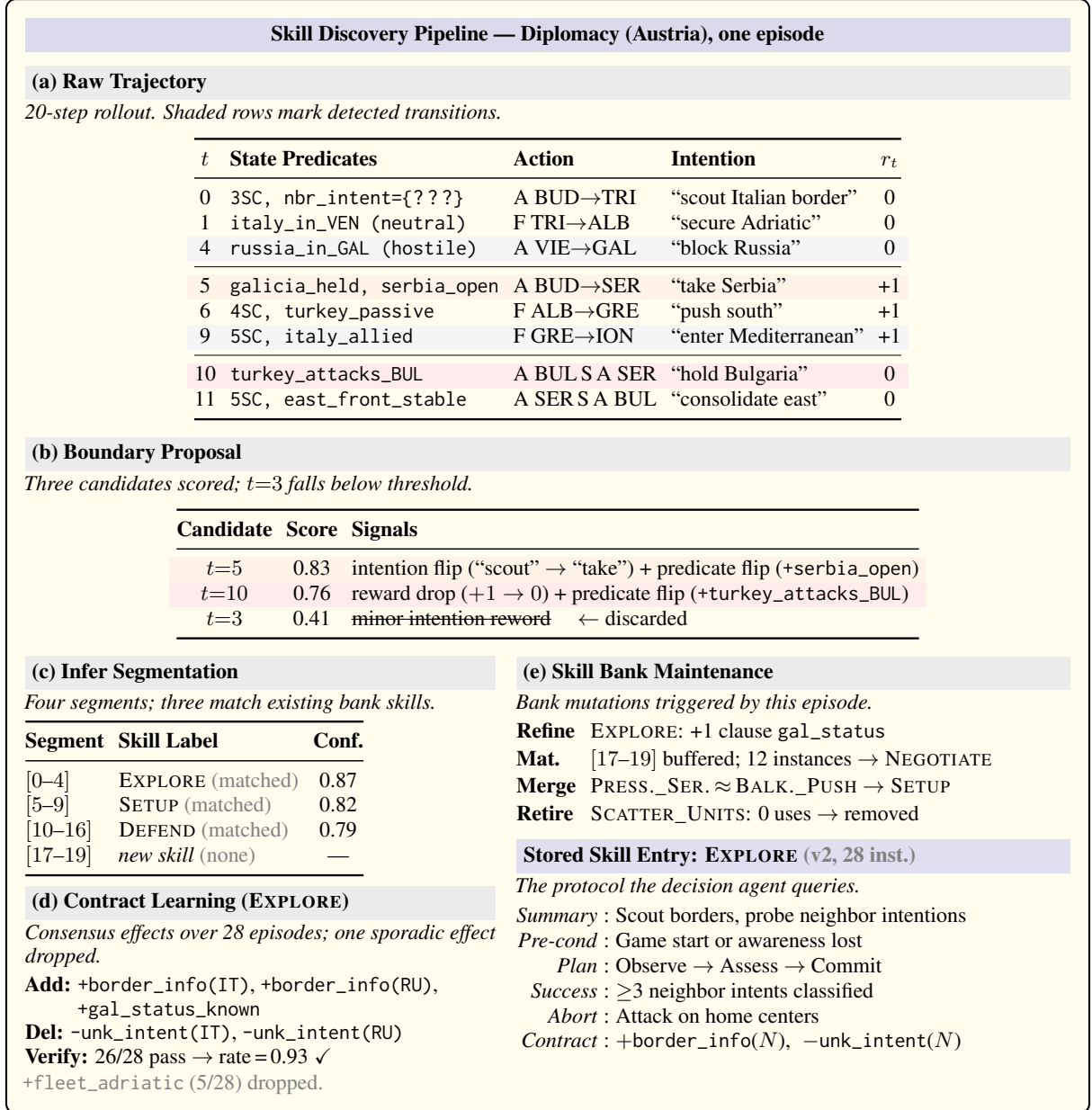


Figure 2: **Skill bank agent pipeline on one Diplomacy episode (Austria).** (a) **Raw Trajectory.** A decision-agent rollout; shaded rows mark skill transitions. (b) **Boundary Proposal.** We score each timestep for transition signals and discard low scorers. (c) **Infer Segmentation.** We select true boundaries and label each segment with a bank skill or *new skill*. (d) **Contract Learning.** We aggregate state deltas across all instances of EXPLORE and drop sporadic effects to obtain a verified contract. (e) **Skill Bank Maintenance.** Four operations (refine, materialize, merge, retire) update the bank. The bottom-right box shows the stored protocol the decision agent queries.

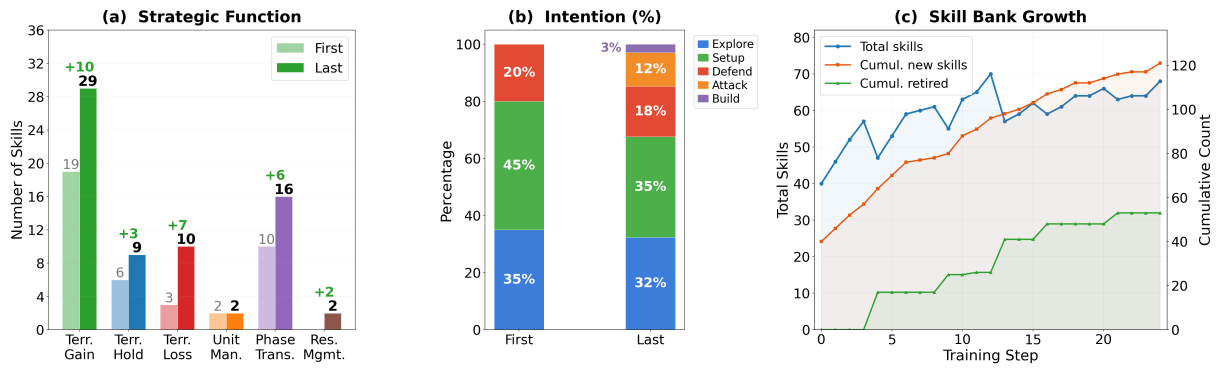


Figure 3: **Skill bank evolution over Diplomacy training.** (a) **Development of Strategic Function Categories** from the first to the last training step. Compared with the initial skill bank, the final bank shows notable increases in phase transition and territory loss skills, indicating a broader tactical repertoire. (b) **Changes in Intention Composition** between the first and last training steps, suggesting increasingly goal-directed behavior and greater diversity across skill categories. (c) **Skill Bank Growth** during training: the active bank stays at roughly 55–70 skills, while 121 are discovered overall. Periodic curation removes 53 redundant skills via merge and split, keeping the bank compact and effective.

Cell	Ablation	Setting	Reward $\uparrow$	Δ vs. A0	Peak $\uparrow$	Final Skills
A0	Full system, 5 LoRAs	Full co-evolution for 10 steps	544.62 ± 20.44	–	578.00	6
A1	Freeze skill-selection LoRA	–	558.88 ± 7.62	+14.25 (+2.6%)	565.12	7
A2	Freeze action-taking LoRA	–	512.54 ± 23.44	–32.08 (–5.9%)	561.12	7
A3	Freeze segmentation LoRA	–	578.83 ± 44.31	+34.21 (+6.3%)	636.75	6
A4	Freeze contract-learning LoRA	–	486.58 ± 59.69	–58.04 (–10.7%)	561.12	5
A5	Freeze curator LoRA	–	549.12 ± 32.75	+4.50 (+0.8%)	580.62	8
A6	Merge decision LoRAs, 2 $\rightarrow$ 1	–	528.62 ± 5.88	–16.00 (–2.9%)	534.00	7
A7	Merge skill-bank LoRAs, 3 $\rightarrow$ 1	–	527.08 ± 24.56	–17.54 (–3.2%)	551.88	6
A8	Merge all LoRAs, 5 $\rightarrow$ 1	–	493.21 ± 21.62	–51.42 (–9.4%)	560.62	5
B1	5 LoRAs, rank = 4	Parameter-matched low-rank split	500.17 ± 22.12	–44.46 (–8.2%)	553.12	8
B2	1 LoRA, rank = 80	Parameter-matched merged adapter	530.92 ± 14.94	–13.71 (–2.5%)	548.38	7
C1	Remove skill-following reward	$w_{\text{follow}} = 0$	558.75 ± 19.62	+14.12 (+2.6%)	581.88	7
C4	Zero curator gradient	Stop gradient for curator reward	550.33 ± 17.56	+5.71 (+1.0%)	563.12	6
C5	Environment reward only	Remove auxiliary reward terms	505.58 ± 21.44	–39.04 (–7.2%)	547.25	6
E1	Decision-only	Skill bank frozen; Phase B skipped	520.04 ± 16.06	–24.58 (–4.5%)	570.62	0
E2	Skill-bank-only	Decision agent frozen	528.50 ± 3.25	–16.12 (–3.0%)	550.38	5
F0	Freeze bank at step 0	Train decision agent for 5 steps with initial SFT cold-start bank	555.96 ± 28.00	+11.33 (+2.1%)	579.38	–
F2	Freeze bank at step 2	Train decision agent for 5 steps with A0 step-2 bank	566.79 ± 29.38	+22.17 (+4.1%)	587.12	–
F5	Freeze bank at step 5	Train decision agent for 5 steps with A0 step-5 bank	586.42 ± 16.81	+41.79 (+7.7%)	597.62	–
F8	Freeze bank at step 8	Train decision agent for 5 steps with A0 step-8 bank	562.83 ± 26.12	+18.21 (+3.3%)	582.62	–
F9	Freeze bank at step 9	Train decision agent for 5 steps with near-final A0 bank	609.50 ± 38.38	+64.88 (+11.9%)	652.50	–

Table 11: **Fine-grained LoRA, reward, and co-evolution ablations on CANDY CRUSH.** We report the mean reward over the last three training iterations with 95% confidence intervals, the change relative to the full five-LoRA system, peak reward, and the final number of active skills. Freezing contract learning, merging all adapters, reducing per-adapter rank, and using environment reward only all substantially degrade performance, suggesting that role-specific LoRA decomposition and shaped rewards are important for stable co-evolution. The decision-only and skill-bank-only variants also underperform the full system, supporting the need to co-evolve both agents. The freeze-at-iteration rows further show that banks saved from later co-evolution checkpoints remain useful when frozen, with the near-final frozen bank achieving the strongest reward.

I Training Curve

Figure 4 presents the training reward curves for all six game environments. In the single-player games (2048, Candy Crush, Tetris, and Super Mario), the ego-player reward increases steadily over co-evolution steps, confirming that the joint optimization of the decision agent and skill bank yields progressively stronger play. The multi-player games (Avalon and Diplomacy), trained under self-play, exhibit flatter reward trajectories. This is expected: since all players share the same model, improvements to the ego agent are mirrored by equally stronger opponents, causing the reward to converge near the game-theoretic equilibrium rather than rise monotonically. Notably, the variance bands in Diplomacy remain tight throughout training, suggesting stable learning dynamics despite the complex 7-player negotiation setting. The true policy improvement in these competitive games is revealed not by the self-play reward curve but by evaluation against external opponents, as reported in Table. 1.

J R10: Robustness to state-summary noise.

To address reviewer concerns about the sensitivity of COS-PLAY to predicate extraction errors and textual state-summary noise, we conduct an inference-time robustness study on CANDYCRUSH. This experiment tests whether the learned policy and skill bank rely on brittle string-level matching of compact state summaries, or instead capture more distributed semantic cues that remain useful under corrupted observations. We fix the paper best checkpoint and skill bank, without any retraining, and evaluate five conditions over 16 episodes with the same decoding setup. The control condition uses the original state summary, while the four perturbed conditions randomly drop key-value fields, drop half of the fields, remove the entire board field, or inject $\pm 20\%$ noise into numeric values. The perturbations are implemented through an environment-variable-gated hook in the state-summary pipeline, so the default execution path is unchanged when no noise specification is provided.

The results show that all four noise settings remain within the 95% confidence interval of the no-noise control. Even under aggressive corruption, performance changes are small: dropping 25% of summary fields reduces reward by 10.8%, dropping 50% of fields slightly increases reward by

4.7%, removing the full board field reduces reward by only 2.0%, and numeric perturbation reduces reward by 3.1%. These results suggest that the action-taking LoRA and skill bank do not depend on exact summary-string templates or a single critical field. Instead, COS-PLAY appears to use redundant semantic cues distributed across the state representation and learned skills, making it robust to moderate perception and predicate-extraction noise.

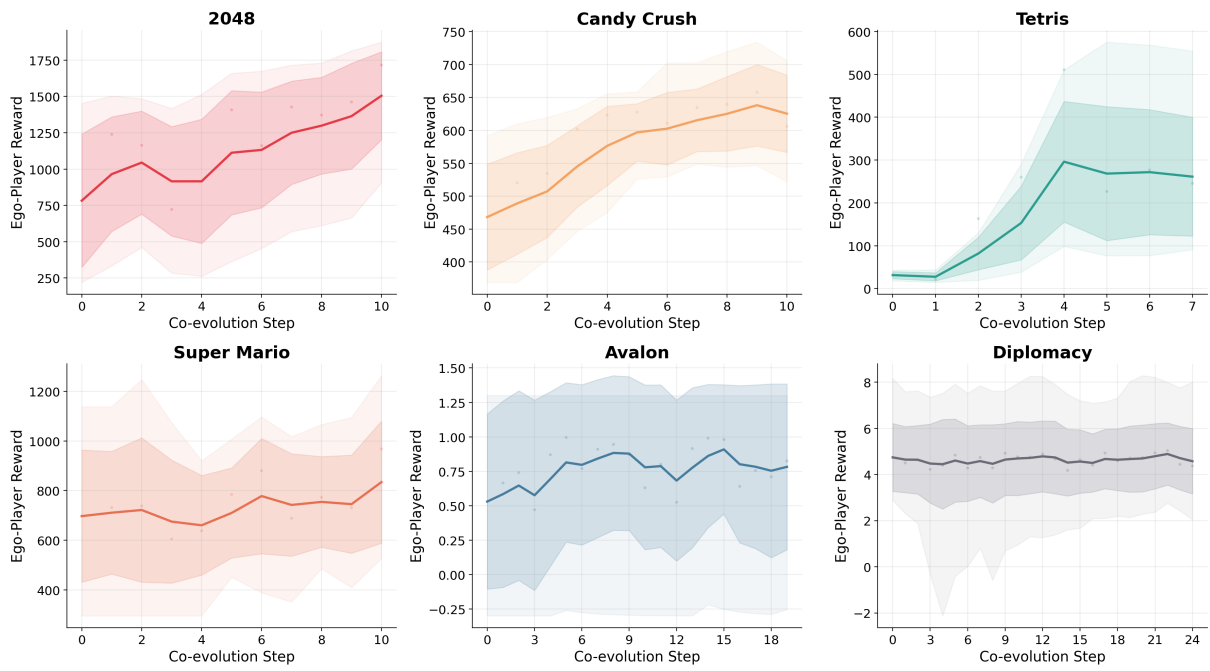


Figure 4: **Co-evolution reward curves for all games.** Single-player games show steady gains, indicating improved strategies from joint decision-agent and skill-bank training. Multiplayer self-play remains flat because all players improve symmetrically, pushing rewards toward equilibrium.

Cell	Ablation	Setting	Reward $\uparrow$	Δ vs. N0	Min	Max
N0	No noise control	Original compact state summary	388.19 ± 47.43	–	277	582
N2	Field dropout, $p = 0.25$	Randomly drop 25% of key-value fields from the state summary	346.38 ± 37.28	-41.81 (–10.8%)	216	468
N3	Field dropout, $p = 0.50$	Randomly drop 50% of key-value fields from the state summary	406.38 ± 56.52	$+18.19$ (+4.7%)	201	592
F-board	Remove board field	Drop the full board field from the state summary	380.56 ± 35.03	-7.63 (–2.0%)	302	493
NUM	Numeric perturbation	Multiply numeric fields by $1 \pm U(0, 0.2)$	376.00 ± 42.64	-12.19 (–3.1%)	240	510

Table 12: **R10 state-summary noise robustness study on CANDY CRUSH.** We evaluate whether COS-PLAY is sensitive to predicate extraction and compact textual state-summary noise. The experiment is inference-only: we fix the paper best CANDYCRUSH action-taking adapter and skill bank, without retraining, and run 16 episodes per condition with max steps 50, temperature 0.3, and seed 42. Noise is injected through an environment-variable-gated summary hook after `get_state_summary()` produces a compact key=value representation. All four noisy conditions remain within the 95% confidence interval of the no-noise control. Even removing the full board field reduces reward by only 2.0%, and numeric perturbation reduces reward by only 3.1%, suggesting that the action-taking LoRA and skill bank rely on distributed semantic cues rather than brittle string-level matching.

K Qualitative Analysis

To complement the aggregate results, we present step-level case studies in Candy Crush and Diplomacy that show how the evolved skill bank changes trajectory-level decisions across stages of play (Figures 5–8). We include strategy annotations and brief justifications explaining why COS-PLAY outperforms GPT-5.4 at key steps. We also provide failure analysis for both GPT-5.4 and COS-PLAY in Figure 9, highlighting qualitative differences and key insights. Finally, we examine skill retrieval and its causal role in Diplomacy in Figure 10, showing how retrieved skills shape phase transitions and subsequent actions.

Candy Crush Strategy Comparison: GPT-5.4 vs. COS-PLAY

Setting		
Game: Candy Crush	Episodes: GPT-5.4 median vs. our best (+47%)	Final Outcome:
Evolved Skill Bank (2 of 6 skills deployed):		
OPTIMIZE	Removes inefficiencies, clears blockers, restructures the board.	<i>Protocol:</i> Analyze → prove → verify.
CLEAR	Harvests the best available match by exploiting a prepared board state.	<i>Protocol:</i> Identify → execute → assess.
Key Distinction: OPTIMIZE prepares the board; CLEAR harvests it. GPT-5.4 uses ATTACK ≈ EXECUTE) with no clear phase boundaries.		
Aggregate Contrast: COS-PLAY averages 6.6 steps per skill run with 7.6 switches. GPT-5.4 averages 10.2 steps per run with 14.0 switches. Best single-step reward: 62 (prepared cascade).		
Early Game (Steps 0–10)		
GPT-5.4		
Step 0 – Opening: [CLEAR] Swaps ((5,4),(5,5)) for an immediate central G match.		
Reward: +15, cumul.=15		
Strategy: Takes the first available central match for 15 points; the board is no better off afterward.		
Step 3 – Minimal match: [CLEAR] Swaps ((2,4),(2,5)) for a vertical red 3-match.		
Reward: +3, cumul.=33		
Strategy: A bare 3-match—no cascade, no board restructuring.		
Step 5 – Label switch: Already switched CLEAR→ATTACK by step 4.		
Reward: +15, cumul.=58		
Strategy: Labels keep changing, but no actual preparation happens between harvests.		
Step 0 – Opening: row has 4 P's. Targets the flag, not the 3-point match. Reward: +17, cumul.=32. Why better: COS-PLAY targets cascades; GPT-5.4 does not.		
Step 3 – Setup: State: note=Swapped. Prior restructuring #18→((5,3),(5,4)). Reward: +34, cumul.=66. Why better: OPTIMIZE sets up a 3-point match.		
Step 5 – Opening: skill switch after note=Swapped. Reward: +13, cumul.=79. Why better: COS-PLAY already restructured by step 4 with n		

Figure 5: Step-level comparison between GPT-5.4 and our method in Candy Crush (1/2: Setting & Early Game). Continued in Figure 6.

Candy Crush Strategy Comparison (cont'd)

Mid Game (Steps 11–34)

GPT-5.4

Step 23 – Reactive midgame: Still in *[CLEAR]*, following the generic “immediate match and cascade setup” approach.

Reward: +6, cumul.=277

Strategy: Still in reactive mode since step 22; no restructuring to set up step 28’s cascade.

Step 28 – Missed jackpot: *[CLEAR]* “Complete the bottom-right candy match” ((6,6),(6,7)).

Reward: +3, cumul.=327

Strategy: Picks a corner match for 3 points; a higher-value cascade is available elsewhere on the board.

COS-PLAY

Step 23 – OPTIMIZE cycle: First step of the mid-game OPTIMIZE phase. State: note=Swapped G/P to clear threat, +11.

The previous CLEAR phase earned a steady 10–31 per step; the agent now switches back to preparation, setting up step 28’s cascade.

Reward: +25, cumul.=355 (vs. 277)

Why better: Back in preparation while GPT-5.4 stays in CLEAR. Lead: +28%.

Step 28 – Cascade jackpot: *[OPTIMIZE]* 6th step of this phase. State: note=Three new pairs created, threat of chain reaction.

The previous five OPTIMIZE steps yielded modest 7–24 rewards while patiently restructuring the board. Now #9→((3,5),(4,5)) triggers the episode’s largest payoff.

Reward: +62, cumul.=455 (vs. +3, cumul.=327)

Why better: Five restructuring steps turn the flagged “chain reaction” into a 62-point cascade—20× GPT-5.4’s reward at this step. Gap: +39%.

Late Game (Steps 35–49)

Step 40 – Endgame panic: Switches to *[EXECUTE]*: “cut pairs fast with 10 moves left.”

Reward: +21, cumul.=455

Strategy: Switches to EXECUTE under time pressure. Score: 434 vs. COS-PLAY 573.

Step 46 – Broken intent: *[EXECUTE]* “maximize endgame points.”

Reward: +3, cumul.=509

Strategy: States “maximize endgame points” but scores only +3.

Step 49 – Final move: *[EXECUTE]* “horizontal four-match.”

Reward: +10, final score=547

Step 40 – Endgame OPTIMIZE: The skill description adapts to “maximize remaining moves” (compared to the mid-game’s “maximize cascades”). State: note=Created 3 new pairs.

Reward: +24, cumul.=637 (vs. 455)

Why better: Score: 573 vs. 434 (+32%) with the same 10 moves left. OPTIMIZE keeps preparing; GPT-5.4 switches to EXECUTE. Lead: +40%.

Step 46 – CLEAR + URGENCY: The system injects an urgency signal: “very few moves left—maximise every action.” Preceding steps scored +26, +21, +26.

Reward: +30, cumul.=760 (vs. +3, cumul.=509)

Why better: With the URGENCY signal, the agent sustains 21–30 points per step; GPT-5.4 scores +3 despite its own “maximize endgame” label.

Step 49 – Final move: *[CLEAR]* + URGENCY. #9→((4,4),(5,4)).

Reward: +22, final score=806 (vs. +10, 547). +47% improvement.

Figure 6: Step-level comparison between GPT-5.4 and our method in Candy Crush (2/2: Mid & Late Game). See Figure 5 for setting and early game.

Diplomacy Strategy Comparison: GPT-5.4 vs. COS-PLAY

Setting

Game: *Diplomacy (Austria)* **Episodes:** *COS-PLAY best vs. GPT-5.4 best & typical* **Final Outcome:** *COS-PLAY: 7 SC vs. GPT-5.4: 7 SC (best), 4 SC (typical)*

Center Trajectories:

COS-PLAY 3→3→3→3→4→4→4→4→4→5→5→5→6→6→6→7→7→7→7→7
GPT-5.4 (best) 3→3→5→5→5→5→5→5→5→5→5→5→5→5→5→7→7→7→7→7
GPT-5.4 (typ.) 3→3→3→4→4→4→3→3→3→3→3→4→4→4→4→4→4→4→4

Evolved Skill Bank (4 skills deployed):

EXPLORE	Scout borders and probe neighbor intentions.	<i>Protocol:</i> Observe → assess → commit.	Steps 0–4
SETUP	Secure supply centers, prepare for expansion.	<i>Protocol:</i> Reposition → coordinate → strike.	Steps 5–10, 17–19
DEFEND	Protect holdings after territorial growth.	<i>Protocol:</i> Consolidate → support → hold.	Steps 11–16
ATTACK	Aggressive pivot when SETUP yields no growth.	<i>Protocol:</i> Target → commit → exploit.	Steps 11–14 (3/28 ep.)

Key Distinction: EXPLORE → SETUP → DEFEND forms a strict temporal pipeline. GPT-5.4 uses five overlapping labels (*Pressure Serbia* ≈ *Secure Balkan* ≈ *Defensive Line*) with no clear phase boundaries.

Aggregate Contrast: COS-PLAY averages **3 phase transitions** per episode with **2–3 switches**; GPT-5.4 uses 8+ intention tags with a switching rate of ~0.43. The stability floor for COS-PLAY is min=3 SC (it never loses starting centers), compared to GPT-5.4's min=1 SC (near-elimination in 27% of episodes).

Early Game (Steps 0–4)

GPT-5.4

Step 0 – S1901M, Opening: [*POSITION*] Textbook Balkan opening: A BUD–SER, A VIE–GAL, F TRI–ALB.
Effect: Immediate Serbia grab + Galicia contest + Albania staging.
Strategy: Commits all three units to the Balkans on turn 1—fast but inflexible, with no reconnaissance.

Steps 2–3 – S1902M, 5 SC: Already at **5 centers and 5 units** by step 2.

Orders: A BUD–RUM, A SER S A GRE–BUL, F GRE–BUL/SC.

Effect: All-in Balkan push—aggressive three-army surge.

Strategy: All-in on one theater; if opponents push elsewhere, there is nothing left to respond with.

Summary: Reaches 3 → 5 by step 2, but the lack of flexibility sets up the 13-phase plateau that follows.

COS-PLAY

Step 0 – S1901M, Opening: [*EXPLORE*] Scout borders. Orders: A BUD–TRI (not the standard BUD→SER grab).
Effect: The army moves to Trieste to test Italian intent instead of rushing Serbia.

Why better: Gives up the immediate Serbia grab for information. If Italy is hostile (moving to TRI), this blocks them; if friendly, Austria pivots south next turn.

Steps 3–4 – F1902M, 3→4 SC: [*EXPLORE*] Fourth consecutive scouting step. A BUD–RUM: a delayed Balkan push that gains the first new center in Fall 1902.

Effect: Reaches 4 centers—one turn behind GPT-5.4's pace, but with superior positional information.

Why better: Three EXPLORE steps identify Russia (not Turkey) as the better expansion target. The 1-SC delay avoids the overcommitment that traps GPT-5.4 for 13 phases.

Phase transition at step 5: EXPLORE→SETUP triggers at exactly step 5 ($\mu=5.0$, $\sigma=0.0$ across all 28 episodes). Five turns of observation before acting, regardless of center count (3–5).

Figure 7: **Step-level comparison between GPT-5.4 and our method in Diplomacy as Austria** (1/2: Setting & Early Game). Continued in Figure 8.

Diplomacy Strategy Comparison (cont'd)

Mid Game (Steps 5–13)

GPT-5.4

Step 9 – F1903M, 5 SC: [DEFEND] Orders: A BUD–VIE, A RUM–SEV, F AEG–ION.

Effect: Attacks Sevastopol and the Ionian Sea simultaneously—both fail, and Rumania is lost the following turn.

Strategy: Spreads across three fronts at once. The center count stays at 5 for **13 consecutive phases**.

Step 12 – S1904M, still 5 SC: [DEFEND] Orders: A BUD S A GAL–RUM, A GAL–RUM, A BUL H, F AEG H.

Effect: Four hold/support orders with zero territorial gain, all effort directed at retaking Rumania.

Strategy: *Secure Balkan Frontier* is selected for 14 of 15 steps, providing no signal to change approach.

COS-PLAY

Step 9–10 – F1904M→W1904A, 4→5 SC: [SETUP] Orders: F APU S F ION—fleet cooperation in the Adriatic and Ionian, establishing a persistent naval coordination chain.

Effect: Centers steadily grow to 5 through sustained mutual support.

Why better: Instead of spreading thin, COS-PLAY repeats one coordination pattern (F APU S F ION, in 12 of 20 steps) that locks down the Adriatic–Ionian corridor.

Step 12–13 – F1905M→W1905A, 5→6 SC: [DEFEND] The SETUP→DEFEND transition triggers after expansion. F APU S F ION continues—consistent naval support secures the 6th center.

Effect: Steady growth while GPT-5.4 remains stuck at 5 for 13 phases.

Why better: SETUP→DEFEND acts as a “stop expanding, hold what you have” gate. GPT-5.4 has no such boundary and tries to attack and defend at the same time.

Late Game (Steps 14–19)

GPT-5.4

Step 14–15 – F1904M→W1904A, 5→7 SC: [ATTACK] Orders: F AEG S A BUL–CON, A BUL–CON.

Effect: Finally breaks through to Constantinople after 12 turns of stagnation.

Strategy: The breakthrough comes from Turkey collapsing, not from a change in GPT-5.4's play.

Step 18–19 – F1905M, 7 SC: Orders: A BUL–RUM, A CON–SMY, F AEG S A CON–SMY.

Effect: Continues pressing Turkey but achieves no further gains.

Final: 7 SC. All growth came in one late burst (5 → 7 at step 15).

COS-PLAY

Step 15–16 – F1906M→W1906A, 6→7 SC: [DEFEND] Orders: F APU S A TYR–VEN—a decisive Venice strike supported by the persistent Adriatic fleet chain, with A BUD B built for consolidation.

Effect: The 7th center is secured through a coordinated multi-unit operation.

Why better: Venice falls because Tyrolia (positioned since EXPLORE) and the Adriatic fleet chain converge—a multi-phase setup that GPT-5.4's single-skill system cannot reproduce.

Step 17–19 – S–F 1907M, 7 SC: [SETUP] Late-game repositioning. F APU S F ION continues, and no center is ever lost.

Final: 7 SC. Steady growth (3 → 4 → 5 → 6 → 7), one gain per game stage. **Zero centers lost at any point.**

Key Findings

COS-PLAY scouts for four turns before committing; GPT-5.4 rushes to 5 SC by step 2 and then stalls for 13 phases. Three skill phases (EXPLORE→SETUP→DEFEND→SETUP) replace 8+ intention tags (switching rate ~0.43). Stability floor: min=3 SC versus GPT-5.4's min=1 SC (16 of 60 episodes end at ≤ 3). One repeated naval pattern (F APU S F ION, 12 of 20 steps) outperforms GPT-5.4's scattered fleet orders.

Figure 8: Step-level comparison between GPT-5.4 and our method in Diplomacy as Austria (2/2: Mid & Late Game). See Figure 7 for setting and early game.

Diplomacy Failure Analysis: Stagnation vs. Collapse

COS-PLAY Failure Mode: “Stuck in Support Loop” (5/28 episodes end at 3 SC)

Episode	Power	Most-Repeated Action	Repeat Rate
diplomacy_c40f3dc3	Italy	F NAP S A ROM (14/20 steps)	42%
diplomacy_2b255046	Italy	F NAP S A ROM (15/20 steps)	47%
diplomacy_95bd25f4	Italy	A APU H (9), F NAP S A ROM (7)	53%
diplomacy_e66fffd8e	Austria	F ALB S A VEN-TRI (13/20 steps)	37%
diplomacy_c5f6577c	Germany	A BER-MUN (5), F BAL S A RUH-KIE (5)	21%

Root Cause: The model has a strong **action-1 bias**: action #1 is selected in 85% of all steps (90% in failure episodes vs. 82% in successes). Action 1 is usually a SUPPORT order, so failing episodes get stuck supporting the same unit over and over. Skill transitions still fire (EXPLORE→SETUP→DEFEND), but the action adapter does not vary its orders enough to break out.

GPT-5.4 Failure Mode: “Collapse” (16/60 episodes, 27%, end with ≤ 3 SC)

Episode	Center Trajectory	Failure Pattern
episode_010	3→5→5→5→4→4→2→2→1→1	Peaked step 2, continuous decline for 17 steps
episode_028	3→5→5→5→5→3→3→3→2→2	Peaked step 2, lost 3 centers mid-game
episode_043	3→4→4→4→4→4→3→3→2→2	Slow bleed from mid-game onward

Root Cause: *Secure Defensive Line* stays active for 12–15 consecutive steps, with the intention cycling SURVIVE→DEFEND→SURVIVE. Centers keep dropping, but the skill bank has **no recovery skill** for losing positions. Once the decline starts, there is no way to pivot—the defensive skill orders retreats and disbands that accelerate the loss.

Key Insight & Qualitative Outplay

Stagnation vs. Collapse: COS-PLAY fails by **stagnation**—stuck at 3 SC, never growing. GPT-5.4 fails by **collapse**—grows to 4–5 SC, then drops to 1–2. Stagnation is the safer failure mode: the agent never loses its starting centers. The phased skill structure provides a *floor*—even with poor action selection, the agent does not abandon centers (min=3 vs. GPT-5.4 min=1).

Outplay Example (France, Step 8, F1904M): France holds only 3 SC but has units in aggressive positions: A TYR (deep in Austrian territory), A BEL, F MAO. COS-PLAY plays A BEL S F NTH-HOL—a joint strike with England—taking **3 new centers in one turn** (3 → 6). The preceding 8-turn “holding pattern” (BRE↔MAO cycling) made GPT-5-mini opponents commit elsewhere; the simultaneous strike went unanswered. At the same stage, GPT-5.4’s best Austria episode sits at 5 SC with four hold/support orders and *Secure Balkan Frontier* selected for 14 of 15 steps.

Figure 9: **Failure analysis for both methods in Diplomacy.** COS-PLAY fails by stagnation (5/28 episodes plateau at 3 SC); GPT-5.4 fails by collapse (16/60 decline to 1–2 SC). Stagnation is the safer failure mode.

Diplomacy: Skill Retrieval & Causal Mechanism

Skill Retrieval Stress Test

COS-PLAY: Clean Phase Boundaries

Transition	Trigger	Step (μ)	N
EXPLORE→SETUP	Temporal boundary	5.0 ± 0.0	28/28
SETUP→DEFEND	Reward ≥ 1.83	11.6 ± 1.2	14/28
SETUP→ATTACK	Low centers (3–4)	13.0 ± 1.0	3/28
DEFEND→SETUP	End-game reposition	17.5 ± 0.8	13/28

Pattern: EXPLORE fires only at 3 SC (opening). ATTACK fires only at 3–4 SC (desperation pivot). DEFEND spans the full 3–7 SC range. The model uses **step number**, not center count, as its primary switch signal—transitions are time-driven.

GPT-5.4: Noisy, State-Independent

Transition	Steps	Centers	N
Secure Balkan→Def. Line	2–19	2–8	184
Def. Line→Secure Balkan	3–19	2–8	145
Pressure Serbia→Secure Balkan	1–19	3–6	61
Def. Line→Fortify Fronts	3–19	3–8	30

Pattern: The top two transitions are inverses, creating a “ping-pong” loop (Balkan→Def.→Balkan→...) across *all* step and center ranges with no consistent trigger. Confidence scores are uniformly low (0.23–0.24)—the retriever cannot tell when each skill applies. POSITION, DEFEND, and ATTACK all appear across the full center range (2–8), with no specialization.

Causal Mechanism: Skills → Temporal Structure → Action Diversity → Better Outcomes

Factor	Success ($c \geq 5$)	Failure ($c \leq 3$)	Mechanism
Action repeat rate	22%	40%	Fewer repeats = more action-space exploration
Action #1 selection	82%	90%	Failures nearly degenerate—always pick first option
Skill transitions	2–3	2–3	Same temporal schedule fires for both
First center gain	Step 3–9	Never	Early growth compounds; stagnation is self-reinforcing

Why skills help: (i) They impose *temporal structure*: the EXPLORE→SETUP boundary at step 5 is a hard “stop observing, start acting” cutoff that breaks support loops. (ii) SETUP *shifts the action distribution*: the subgoal prompt changes from “scout intentions” to “secure supply centers,” which is enough to move past purely defensive orders. (iii) Skills provide a *floor, not a ceiling*—they prevent collapse (min=3 vs. GPT-5.4 min=1) even with poor action selection, because the phased structure stops the agent from abandoning centers.

Figure 10: **Skill retrieval patterns and causal mechanism in Diplomacy.** Skills function as a curriculum schedule for action exploration, as they impose temporal structure that broadens the action distribution and establishes a safety floor.